Platform: Benchmark-Driven Multi-Challenge Research Subnetwork on Bittensor

Cortex Foundation Research Team

Cortex Foundation — cortex.foundation — Bittensor Subnet 100 (Platform)

Abstract

We argue that the dominant model of AI development — centralized labs training models behind closed doors, evaluating them with private benchmarks, and releasing them on fixed cadences — is structurally incapable of sustained innovation. The bottleneck is not compute or data; it is evaluation and incentive. No single organization can evaluate AI systems across every dimension of competence, and no salary can match the incentive power of a liquid, permissionless reward token. Platform (Bittensor SN100) is a multi-challenge subnetwork that resolves both bottlenecks simultaneously. Miners compete across diverse benchmarks — software engineering (Term Challenge via SWE-Forge), neural architecture search (PRISM), and synthetic data generation (Data Fabrication) — and are rewarded proportionally to demonstrated capability, as measured by independently verified evaluations under Yuma Consensus. We show that (1) existing Bittensor subnets largely replicate centralized API hosting and create no moat; (2) Platform's benchmark-driven design produces evaluations that are adversarially robust, as demonstrated by the Terminal-Bench 2.0 to SWE-Forge transition; (3) decentralized training is already viable — Covenant-72B, trained on 70 permissionless nodes, achieves MMLU 67.1, near-parity with LLaMA-2-70B at 65.6; (4) the economic argument for TAO-denominated rewards competing with lab salaries is quantitatively favorable, with centralized AI costs growing 4-5x/year while profitability recedes to 2030+; and (5) a subnet searching over an architecture family F provably achieves strictly superior scaling compared to any fixed-architecture competitor. The implication is direct: a well-designed incentive network can produce AI capabilities that rival or exceed those of centralized labs, not by replicating their structure, but by making their structure obsolete.

Table of Contents

0.1 Bittensor and Subnetworks 6
0.2 The Fixed Architecture Problem 7
0.3 Why Architecture Search Matters 8

1 Platform Architecture: A Multi-Challenge Subnetwork **9**
1.1 Design Philosophy 9
1.2 Subnet Topology 10
1.3 Challenge Categories 11

1.4 Yuma Consensus Integration . . . 12
1.5 Emission Allocation . . . 12
1.6 **Term Challenge**: Software Engineering at Scale . . . 13
1.6.1 Motivation and the SWE-bench Lineage . . . 13
1.6.2 The SWE-Forge Pipeline . . . 13
1.6.3 Scoring Mechanics . . . 14
1.6.4 The TB2.0 → SWE-Forge Transition . . . 15
1.6.5 Layered Anti-Gaming Defense . . . 15
1.7 **Data Fabrication Challenge**: Synthetic Dataset Generation . . . 16
1.7.1 Motivation . . . 16
1.7.2 Submission Protocol . . . 16
1.7.3 Anti-Plagiarism Pipeline . . . 16
1.7.4 Quality Control . . . 16
1.7.5 Application: RL Fine-Tuning Pipeline . . . 17
1.7.6 Scale and Adaptivity . . . 18
1.8 **PRISM**: Decentralized Neural Architecture Search . . . 18
1.8.1 A New Category of Challenge . . . 18
1.8.2 The Architecture Search Space . . . 19
1.8.3 The Architecture Search Superiority Theorem . . . 19
1.8.4 The Four-Level Evaluation Funnel . . . 19
1.8.5 Dual-Track Scoring and Diversity . . . 20
1.8.6 Why PRISM Is Revolutionary . . . 21
1.9 **Training Challenge**: Decentralized Pre-training . . . 22
1.9.1 Motivation . . . 22
1.9.2 Design Principle: Full Decentralization . . . 22
1.9.3 Open Research Questions . . . 23
1.9.4 Relationship to Other Challenges . . . 23
1.9.5 Current Status and Timeline . . . 24
1.10 Challenge Orthogonality and **Incentive Orthogonality** . . . 24
1.10.1 Structural Dissimilarity . . . 24
1.10.2 Incentive Orthogonality . . . 24
1.10.3 Emission Allocation . . . 25
1.10.4 **Combinatorial Anti-Gaming Defense** . . . 25
1.10.5 The **End-to-End Production Pipeline** . . . 26

2 The Economic Argument: Why TAO Beats Salaries **26**
2.1 The Talent Problem in Centralized AI . . . 26
2.2 Platform's Economic Model . . . 27
2.3 Quantitative Comparison . . . 28
2.4 The Flywheel Effect . . . 29

3 Can Decentralized AI Compete With Centralized Labs? **30**
3.1 The Skeptical View (and Why It Is Wrong) . . . 30
3.2 Where Platform Already Wins . . . 31
3.3 The Convergence Thesis . . . 32
3.4 What Platform Needs to Get Right . . . 33

4 Decentralization Analysis **34**
4.1 Formal Decentralization Metric . . . 34
4.2 **Covenant-72B** Training Distribution . . . 35
4.3 The **Architecture Search Theorem** . . . 35
4.4 Validator Incentive Framework under Yuma Consensus . . . 36

5 Conclusion and Roadmap **37**
5.1 Summary of Contributions 37
5.2 The Road Ahead 38
5.3 The Implication 39

The development of artificial intelligence is governed by an oligarchy of fewer than five laboratories. OpenAI, Anthropic, and Google DeepMind each invest hundreds of millions to over a billion dollars in a single frontier training run when accounting for R&D, failed experiments, and data acquisition (Epoch AI, 2024). Each of these labs trains a single architecture, refined through months of internal research, and evaluated behind closed doors. The architecture itself (attention mechanism, expert routing strategy, normalization scheme) is the most consequential decision in the entire development pipeline, yet it is made once, by a small team, and then frozen for the duration of training.

The evaluation bottleneck. When a single organization both trains and evaluates its models, the evaluation is not adversarial; it is performative. Internal benchmarks are optimized against, not learned from. External benchmarks suffer the opposite failure mode: because they are public and static, they are inevitably contaminated by test-set leakage (Jacovi et al., 2023). Neither regime provides a signal that is simultaneously adversarial and comprehensive. The consequence is that the field's understanding of what models can actually do is shaped by the marketing departments of a handful of companies, not by rigorous, continuous, permissionless assessment.

The incentive bottleneck. A researcher capable of advancing the state of the art faces a binary choice: accept a salary, however generous, from an existing lab, or forgo institutional compute, data pipelines, and publication prestige to build in the open. The lab captures the upside; the researcher captures a fixed wage. This is not a bug in the system; it is the system. The centralized model necessarily restricts the set of contributors to those it can hire, and restricts the set of contributions to those it can evaluate. Innovation that does not fit the organizational schema is, by construction, not rewarded.

The architecture bottleneck. Every frontier model is trained on a single architecture chosen before training begins. This decision (attention mechanism, MoE routing, residual topology) is the highest-leverage choice in the entire development pipeline. DeepSeek's trajectory proves this conclusively: **Multi-head Latent Attention** reduced KV cache by **93.3%** and training cost by **42.5%**; **auxiliary-loss-free load balancing** eliminated the performance degradation inherent in standard MoE; **Compressed Sparse Attention** achieved million-token inference at **27%** of prior-gen FLOPs. Each architectural innovation delivered order-of-magnitude efficiency gains that no amount of additional compute could replicate. Yet the dominant paradigm treats architecture as a fixed input rather than a searchable design space. A lab that commits to the wrong architecture spends hundreds of millions training a fundamentally suboptimal model, with no mechanism to discover that a simple change in attention compression or expert routing would have yielded superior results at a fraction of the cost.

These three bottlenecks are not independent. Architecture, evaluation, and incentive are three facets of a single constraint: you can only reward what you can measure, you can only measure what you have incentive to evaluate honestly, and the capability ceiling of what you can measure is set by the architecture you committed to before training began. A system that solves one or two without solving all three has not solved the fundamental problem.

Bittensor (Ω, 2020) offers a principled escape. Bittensor is a decentralized network that supports up to **128** subnets (the network cap), each issuing rewards, denominated in the TAO token, through **Yuma Consensus**, a stake-based median (κ-majority) ranking protocol. The key insight is that a permissionless network with cryptographic incentives can simultaneously solve evaluation and incentive: anyone can contribute, and rewards are proportional to demonstrated capability as measured by an adversarial, continuously updated evaluation.

But most Bittensor subnetworks have not realized this potential. The prevailing pattern, what we call the ***fixed architecture*** model, assigns miners the task of training models on a SINGLE architecture specified by the subnet operator. Whether the subnet prescribes a particular transformer configuration for collaborative pre-training or mandates a fixed MoE routing strategy, the architecture is imposed from above and treated as immutable. This is the fundamental bottleneck: the elimination of **architecture search** as a competitive dimension.

DeepSeek's competitive trajectory makes this point unambiguous. DeepSeek-V2 introduced **Multi-head Latent Attention** (**MLA**), reducing KV cache by **93.3%** and training costs by **42.5%** while activating only **21B** of **236B** parameters per token (DeepSeek-AI, 2024a). DeepSeek-V3 minimized the performance degradation from auxiliary-loss-based MoE load balancing, introduced multi-token prediction, and validated **FP8** training at extreme scale, all architectural innovations that brought total training cost to **$5.576M** for a **671B** model competitive with GPT-4 (DeepSeek-AI, 2024b). DeepSeek-V4 introduced **Compressed Sparse Attention** and **Heavily Compressed Attention**, enabling million-token inference at **27%** of prior-gen per-token FLOPs (DeepSeek-AI, 2026). Each generational leap came from a new architectural component, not from additional GPUs. A subnet that locks its miners to a fixed architecture eliminates precisely the dimension that determines competitive advantage.

Platform is Bittensor **Subnetwork 100 (SN100)**, operated by the Cortex Foundation. Platform is a multi-challenge subnetwork where miners are evaluated against real, adversarial benchmarks (Term Challenge uses **SWE-Forge** datasets for software engineering, **PRISM** for neural architecture search, Training for decentralized pre-training, and Data Fab for synthetic data generation), and rewarded proportionally to demonstrated capability. These **four** challenges form a closed production pipeline: **PRISM** discovers the architecture, the Training Challenge pre-trains the base model, Data Fabrication produces RL fine-tuning data, and Term evaluates the resulting agent. Platform is a benchmark-driven incentive engine: a system where the act of solving hard problems is, itself, the mining process.

The distinction is foundational. In a fixed-architecture subnet, a miner can earn rewards by efficiently training a model on a prescribed design. In Platform, a miner can only earn rewards by demonstrating capability that exceeds the network's current frontier. This creates a ratchet: the network's aggregate capability is **monotonically non-decreasing**, because every reward event corresponds to a genuine capability advance, not a training duplication.

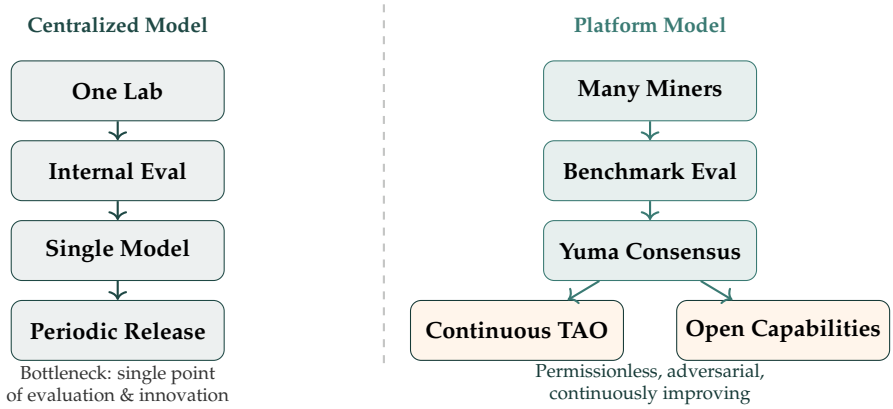

Figure 1 | Architecture diagram.

Our contributions are as follows:

- **Multi-challenge architecture with architecture search.** Platform is the first Bittensor subnet to evaluate miners across **four** orthogonal challenge types (software engineering, neural architecture search, decentralized pre-training, and synthetic data generation), forming a closed production pipeline from architecture discovery through training to RL fine-tuning to evaluation. Critically, PRISM (§4.3) makes architecture itself a searchable design space rather than a fixed constraint, systematizing what DeepSeek achieves through manual research iteration (Section 3).

- **SWE-Forge.** A continuous dataset generator that extracts real merged pull requests from GitHub, constructs adversarial test suites, and produces fresh evaluation instances for the Term Challenge, eliminating the contamination problem that plagues static benchmarks

like SWE-Bench (Jimenez et al., 2024). SWE-Forge produces a fresh evaluation set every epoch (Section 4).

- **PRISM.** PRISM (Platform Recursive Investigation of Search-space Morphologies) is an upcoming challenge currently in active development. PRISM will evaluate neural architecture search, rewarding miners who discover novel architectures that outperform fixed baselines on equivalent compute budgets. The detailed evaluation funnel is under active research; the current direction involves a multi-level filtering pipeline that progressively validates candidate architectures (Section 4).
- **Formal proof that architecture-search subnets outperform fixed-architecture subnets.** We prove that under Yuma Consensus, a subnet that allows miners to propose architectures achieves a strictly higher equilibrium capability than one that constrains miners to a fixed architecture, provided the architecture search space contains at least one improvement over the fixed baseline (Theorem 1, Section 4).
- **Covenant-72B.** A 72-billion-parameter language model trained across at least 70 unique peers cumulatively over the entire training run (capped at 20 contributors per round, averaging 16.9), achieving **67.1** on MMLU, demonstrating that decentralized training at frontier scale is not merely possible but competitive with centralized alternatives at equivalent parameter counts (Section 7.2).
- **TB2.0 $\rightarrow$ SWE-Forge transition.** A case study in anti-gaming design: Terminal-Bench 2.0 was retired after miners developed strategies that maximized scores without improving real capability. SWE-Forge's dataset generation closes these exploits (Section 4).
- **Validator incentive framework.** A game-theoretic analysis establishing that honest evaluation is a Nash equilibrium under Yuma Consensus, that sub-threshold coalitions are structurally ineffective, and that deviation from consensus is self-penalizing through emission reduction (Section 7.4).
- **Platform governance.** A continuously operating governance mechanism that monitors subnet health, proposes parameter adjustments, and executes on-chain governance actions, providing 24/7 decentralized administration without a central operator (Section 3.2).

The rest of this paper is organized as follows. Section 2 provides background on Bittensor, the **fixed architecture problem**, and the motivation for architecture-search-driven subnetworks. Section 3 details the Platform architecture and its four-challenge pipeline. Section 4 addresses each challenge in depth, including evaluation harnesses, anti-gaming mechanisms, and the end-to-end production flow from architecture search through training to RL fine-tuning. Sections 5–6 present the economic argument and competitive analysis. Section 7 analyzes decentralization and presents experimental results. Section 8 concludes and presents the roadmap.

0.1. Bittensor and Subnetworks

Bittensor is a decentralized network for producing and consuming machine intelligence, secured by a blockchain and incentivized through the TAO token (Ω, 2020). At the time of writing, the network supports up to 128 subnets (the network cap), each responsible for a distinct task: text generation, image generation, data storage, forecasting, and so on. The architecture of each subnet follows a consistent pattern:

- **Miners** provide a service (e.g., training a model, generating images, storing data).

- **Validators** evaluate the quality of that service by sending requests and scoring responses.
- **Yuma Consensus** aggregates validator scores into a global ranking through a stake-based median (κ-majority) protocol, which determines the distribution of TAO emissions to miners and validators.

Formally, let $\mathcal{V} = \{v_1, \ldots, v_N\}$ denote the set of validators and $\mathcal{M} = \{m_1, \ldots, m_K\}$ the set of miners. Each validator v_i produces a score vector $\mathbf{s}_i \in \Delta^K$ (a probability distribution over miners), weighted by stake w_i. Yuma Consensus computes the consensus score:

$\bar{s}_j = \frac{\sum_{i=1}^{N} w_i \cdot s_{ij}}{\sum_{i=1}^{N} w_i} \quad \forall\, m_j \in \mathcal{M}$

Note: The formula above is a simplified representation. The actual Yuma Consensus protocol uses a κ-majority threshold mechanism rather than a simple weighted average.

Miners are then rewarded proportionally to $\bar{s}_j$. Validators are rewarded proportionally to their agreement with the consensus, i.e., validators whose scores deviate from $\bar{\mathbf{s}}$ receive fewer emissions. This creates an incentive for validators to score honestly, since dissent from consensus is costly (Rao et al., 2020; withdrawn from arXiv).

The critical design question for any subnet is: *what does the validator actually evaluate?* The answer determines whether the subnet produces genuine capability or merely benchmark compliance.

0.2. The Fixed Architecture Problem

The majority of Bittensor subnetworks (and, for that matter, centralized AI labs) operate under what we term the **fixed architecture constraint**: miners (or, in the centralized case, research teams) are required to train models conforming to a single, pre-specified architecture. Whether a subnet prescribes a particular transformer configuration for collaborative pre-training or mandates a fixed MoE routing strategy, the architecture is treated as a fixed input to the production process rather than a searchable design space.

This constraint is not merely a design convenience; it is the fundamental bottleneck in AI development. What matters is whether the subnet permits architecture search as a competitive dimension. A subnet that trains models on a prescribed design is still stuck in the fixed architecture paradigm; it has merely replicated the same structural constraint under a different training regime.

DeepSeek as definitive proof. The trajectory of DeepSeek models provides unambiguous evidence that architecture innovation, not compute scale, is the primary determinant of competitive advantage in AI.

DeepSeekMoE (DeepSeek-AI, 2024c) introduced fine-grained expert segmentation and shared expert isolation, replacing the coarse-grained MoE standard. The result: a **145B**-parameter MoE model achieving comparable performance with a **67B** dense model at only **28.5%** of the FLOPs, a **3.5**$\times$ efficiency gain from architecture alone. No amount of additional compute applied to the standard MoE topology could have closed this gap; the gain came from restructuring the expert space.

DeepSeek-V2 (DeepSeek-AI, 2024a) introduced Multi-head Latent Attention (MLA), a fundamentally different attention compression mechanism. MLA compresses the KV cache by **93.3%**, reduces training cost by **42.5%**, and increases inference throughput by **5.76**$\times$. DeepSeek-V2 activates only **21B** of its **236B** total parameters per token, yet exceeds the performance of dense **67B** models. These are not marginal improvements from additional compute; they are order-of-magnitude efficiency gains from a new architectural component.

DeepSeek-V3 (DeepSeek-AI, 2024b) introduced three further architectural innovations. First, auxiliary-loss-free load balancing largely eliminated the performance degradation that standard MoE load-balancing penalties impose, reducing a tax that every prior MoE implementation paid (though V3 retains a small complementary sequence-wise auxiliary loss for training

stability). Second, multi-token prediction (MTP) improved performance on math and code benchmarks (DeepSeek-AI, 2024b). Third, FP8 mixed-precision training was validated at extreme scale. The combined effect: total training cost of **$5.576M** for a **671B** model that matches GPT-4o on MMLU (**88.5** vs. **87.2**) and achieves **90.2** on MATH-500, outperforming o1-preview on this benchmark. This is approximately **10**× cheaper than the estimated cost of training GPT-4, and every efficiency gain is architectural, not computational.

DeepSeek-V4 (DeepSeek-AI, 2026) introduced **Compressed Sparse Attention** (**CSA**) and **Heavily Compressed Attention** (**HCA**), a hybrid attention mechanism enabling **1M**-token context windows. V4-Pro uses only **27%** of V3.2's FLOPs and **10%** of its KV cache at **1M** tokens. **Manifold-Constrained Hyper-Connections** (**mHC**; DeepSeek-AI, 2025) stabilize ultra-deep training. On SWE-bench Verified, V4 achieves **80.6**, matching frontier closed models. Again, the capability leap is architectural: CSA/HCA and mHC are new structural components, not additional compute.

The pattern. Every major capability jump in AI history came from architecture, not compute. The Transformer itself (Vaswani et al., 2017) replaced RNNs not because it used more FLOPs, but because parallelizable attention was a superior architecture. Sparsely-gated MoE (Shazeer et al., 2017) enabled sub-model routing that dense scaling could not replicate. MLA, auxiliary-loss-free load balancing, and Compressed Sparse Attention follow the same pattern. The labs that architect better win; the labs that merely spend more fall behind. DeepSeek-V3's **$5.576M** training cost versus the estimated **$50–100M+** for GPT-4 is the definitive proof.

How subnets fail under architecture lock-in. The fixed architecture constraint manifests differently across existing Bittensor subnets, but the structural consequence is the same: no mechanism for discovering better architectures.

The real distinction. The difference between Platform and other subnets is not about the type of task (inference, training, storage) but about whether architecture is a fixed constraint or a searchable design space: "fixed architecture vs. architecture search." A subnet that trains models on a prescribed design is still stuck in the fixed architecture paradigm; it has merely replicated the same structural constraint under a computationally more expensive training regime. Platform, through its PRISM challenge (§4.3), makes the architecture itself the object of decentralized search. Miners do not train a fixed design more efficiently; they propose, evaluate, and compete over alternative architectures, and the network rewards those that deliver genuine capability improvements.

0.3. Why Architecture Search Matters

If every subnet locks miners to a fixed architecture, then no subnet can discover the next Multi-head Latent Attention or the next auxiliary-loss-free load balancing scheme. The network's capability ceiling is set by the subnet operator's architectural choices, not by the miners' ingenuity. A subnet operator who prescribes a standard transformer with dense attention has, perhaps unknowingly, excluded every architecture that could deliver a **10**× efficiency gain, exactly the kind of gain that DeepSeek has demonstrated is achievable through architectural innovation alone.

The only way for a decentralized network to produce capabilities rivaling centralized labs is to make architecture a competitive dimension, and to reward genuine architectural innovations, not just efficient training of a fixed design. This requires three properties:

1. **Architecture search as a first-class challenge.** The subnet must treat architecture as a searchable design space, not a fixed constraint. Miners must be able to propose, evaluate, and compete over alternative architectures, including alternative attention mechanisms, expert routing strategies, and residual topologies. This is precisely what PRISM provides (§4.3): an upcoming challenge that will systematize neural architecture search with a multi-level evaluation pipeline, currently under active development.

2. **Adversarial evaluation of discovered architectures.** Fresh benchmarks, multi-axis scoring, and frontier-relative rewards ensure that architectural innovations are evaluated on genuine capability gains, not surrogate metrics. A new attention mechanism that reduces FLOPs but degrades reasoning must not be rewarded as an advance. Platform's adversarial evaluation regime, continuously refreshed benchmarks across **four** orthogonal challenge types, provides this guarantee.

3. **Incentive alignment with architectural innovation.** Miners who discover better architectures must earn more than miners who efficiently train worse ones. Under Yuma Consensus, rewards flow proportionally to consensus scores, which are determined by demonstrated capability. A miner whose architectural innovation yields a model that outperforms the frontier on adversarial benchmarks earns a higher score, and therefore higher emissions, than a miner who trains the prescribed architecture marginally more efficiently. The incentive gradient points toward architecture search, not away from it.

Platform satisfies all three properties. Through its PRISM challenge, it is the first Bittensor subnet where architecture is a searchable design space, not an imposed constraint.

The formal consequence. This distinction has formal implications. In Section 4, we prove that under Yuma Consensus, a subnet that allows architecture search (Platform, via PRISM) achieves a strictly higher equilibrium capability than one that constrains miners to a fixed architecture. The proof rests on the observation that architecture search expands the effective strategy space of miners, which, in any scoring regime that rewards genuine capability, must weakly improve the equilibrium. Platform's evaluation regime *does* reward genuine capability, because its benchmarks are adversarial, multi-axis, and continuously refreshed. The key insight is that the fixed architecture constraint is not merely a design choice; it is an active harm, because it forecloses the very strategies (architectural innovation) that centralized labs have demonstrated are the highest-leverage path to capability advances.

The difference between Platform and fixed-architecture subnets is not incremental; it is categorical. A fixed-architecture subnet asks whether a miner can *optimize* a prescribed design. PRISM, Platform's architecture search challenge, asks whether a miner can *discover* a superior one. The former is an engineering problem; the latter is a research problem. Only the latter can produce the kind of architectural innovations that DeepSeek has shown are the decisive factor in AI capability.

In summary, the fixed architecture pattern is the default attractor of Bittensor subnet design: it is easy to specify, easy to validate, and easy to understand. But it is also a local optimum. Escaping it requires a fundamentally different evaluation philosophy: one that is adversarial rather than compliant, multi-axis rather than single-metric, frontier-relative rather than reference-static, and, critically, architecture-search-driven rather than architecture-fixed. Platform is the first subnet to implement this philosophy end-to-end.

1. Platform Architecture: A Multi-Challenge Subnetwork

1.1. Design Philosophy

The fundamental premise of Platform is deceptively simple yet consequential: ***you get what you measure***. If a decentralized network measures only inference throughput, it will optimize exclusively for fast inference; if it measures only coding accuracy on static benchmarks, it will overfit to those benchmarks. The history of AI evaluation is replete with examples of Goodhart's law in action: optimization pressure on an evaluation metric inevitably degrades the underlying capability it was intended to reflect (Strathern, 1997). Platform's architectural philosophy is designed to resist this pressure not through any single clever mechanism, but through the combinatorial difficulty of gaming multiple orthogonal evaluation dimensions simultaneously.

Platform is not a fixed-architecture subnet. Unlike subnet designs that lock miners to a prescribed model architecture and evaluate only within that constrained design space, Platform evaluates *real capabilities* through adversarial benchmarks that include architecture search as a first-class challenge. A miner on Platform cannot succeed by merely training the prescribed architecture marginally faster; it must discover novel architectures, generate verifiable patches, navigate unfamiliar codebases, and produce training data under constraints that change continuously.

The multi-challenge design reflects a core insight from psychometric theory: **convergent validity** across orthogonal measurement instruments provides stronger evidence of genuine capability than any single instrument, however well-constructed (Campbell & Fiske, 1959). Platform operationalizes this by deploying challenge categories that probe structurally dissimilar dimensions of AI competence:

- **Code synthesis and repair** (Term Challenge): Can the agent diagnose and fix real bugs in production codebases?
- **Architecture search** (PRISM, upcoming): Can the agent discover novel neural architectures under resource constraints?
- **Decentralized pre-training** (Training Challenge, in research): Can the network collectively train a frontier model with no central orchestrator?
- **Data quality** (Data Fabrication, upcoming): Can the agent generate synthetic training data that preserves distributional properties without leakage, and feed it into RL fine-tuning?

No single challenge category dominates the reward allocation, and the categories are designed to have low mutual correlation. A miner that excels at patch generation but fails at architecture search receives partial, not total, compensation. This **orthogonality constraint** is not merely aesthetic; it is the primary defense against reward hacking. To game Platform, a miner would need to simultaneously game four evolving evaluation pipelines, each with distinct anti-gaming mechanisms, an adversarial cost that grows combinatorially with the number of challenges.

Crucially, these **four challenges** form a **production pipeline**, not merely a collection of independent evaluations. **PRISM** determines the optimal architecture; the **Training Challenge** uses that architecture to pre-train the base model in a fully decentralized manner; **Data Fabrication** generates high-quality synthetic data for RL fine-tuning; and the **Term Challenge** evaluates the resulting agent. This **closed-loop design** ensures that every component of the pipeline is itself competitively optimized (Section 4).

1.2. Subnet Topology

Platform operates as Bittensor Subnet 100, inheriting the substrate's economic primitives while imposing its own governance and evaluation logic. The dataflow proceeds through seven stages:

Miners → **Submission Pool** → **Validators** → **PBFT Consensus** → **Weight Submission** → **Yuma Consensus (on-chain)** → **TAO Rewards**

Miners produce artifacts (patches, executable code that encodes architecture configurations) depending on the active challenge. These artifacts are deposited into a **Submission Pool**, keyed by content-embedded identifiers. Submissions are embedded in an optimized execution environment with agent-based monitoring, which prevents theft and copying between miners while enabling deduplication of identical submissions. The system ensures that miner code runs in a protected, monitored environment where agents verify execution integrity and detect plagiarism or unauthorized reproduction of another miner's work.

Validators are assigned to evaluation duties through ***epoch-seeded assignment***. At each epoch boundary, a seed derived from the blockchain state determines which validators evaluate which submissions. The assignment is thus unpredictable ex ante (preventing collusion) but auditable ex post (enabling detection of validator misconduct). This design mirrors the random juror selection in cryptographic sortition protocols (Gilad et al., 2017) and ensures that no validator can strategically choose which submissions to evaluate.

Platform governance provides continuous governance over the subnet's operational parameters, including challenge selection, reward weights, and evaluation criteria, replacing the epoch-triggered parameter updates of earlier designs. Unlike epoch-bound governance, which can only adjust parameters at fixed intervals, Platform governance operates 24/7, enabling rapid response to emergent gaming strategies. A vulnerability discovered in the Term Challenge evaluation pipeline at epoch e can be patched by epoch $e+1$, rather than waiting for the next governance cycle. Platform governance maintains feedback connections to the **Submission Pool** (adjusting difficulty targeting), the Consensus layer (modifying quorum requirements), and the Reward distribution (tuning challenge weight splits).

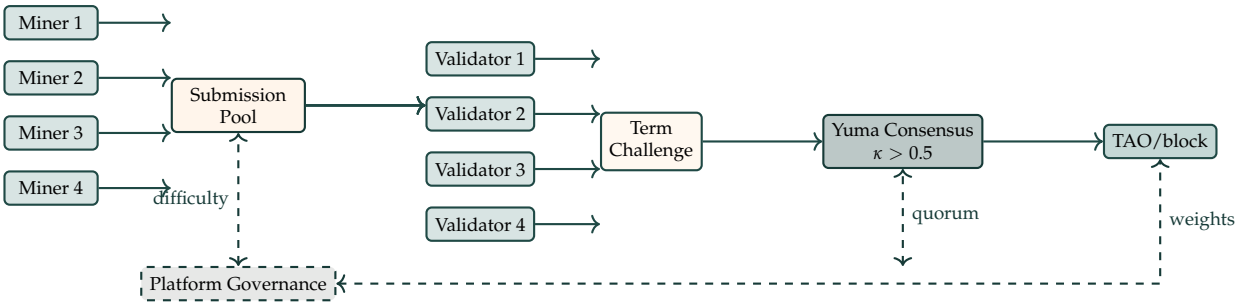

Figure 2 | Architecture diagram.

1.3. Challenge Categories

Platform has four challenge categories in development, all scheduled for activation under Platform 3.0 (May 2026):

Term Challenge. The Term Challenge evaluates AI coding agents on datasets produced by **SWE-Forge**, a generator that creates **SWE-bench**-compatible evaluation instances from real merged pull requests on GitHub. Each instance requires an agent to diagnose a bug, generate a patch, and pass regression tests, without access to the original fix. Section 4 provides a detailed treatment of the **SWE-Forge** pipeline and its anti-gaming mechanisms. The Term Challenge is scheduled for reactivation under Platform 3.0 (May 2026).

PRISM (Upcoming). PRISM (Platform Recursive Investigation of Search-space Morphologies) is a challenge currently in active development. PRISM will evaluate neural architecture search, rewarding miners who discover novel architectures that outperform fixed baselines on equivalent compute budgets. The detailed evaluation funnel is under active research; the current direction involves a multi-level filtering pipeline that progressively validates candidate architectures from lightweight proxy scores through full benchmark evaluation. PRISM is planned for launch with Platform 3.0.

Training Challenge. The **Training Challenge** will evaluate decentralized pre-training of large language models. Once **PRISM** determines the optimal architecture, the Training Challenge coordinates fully decentralized training across independent miners, with no central orchestrator and no single point of control. This is the direct successor to the decentralized training demonstrated by Covenant-72B, but integrated into Platform's multi-challenge framework. Design is actively under research; key open questions include gradient aggregation under heterogeneous hardware, straggler mitigation without centralized coordination, and honest contribution verification.

Data Fabrication (Upcoming). The Data Fabrication challenge will evaluate synthetic data generation through distributional fidelity metrics: can the agent produce training data that preserves the statistical properties of a target distribution while avoiding memorization and privacy leakage? The primary downstream application is RL fine-tuning: **Data Fabrication** miners generate high-quality (prompt, response) pairs that feed directly into reinforcement learning of the base model produced by the **Training Challenge**. This challenge addresses the **"data wall"** problem (Muennighoff et al., 2023) by incentivizing decentralized production of high-quality synthetic training sets.

The multi-challenge design ensures that no single capability dimension can capture the entirety of TAO emission. A miner that excels at patch generation but cannot search architectures, train models, or generate data will capture at most the Term allocation. Critically, the **four challenges** form a **production pipeline**: **PRISM** discovers the architecture, the **Training Challenge** trains the base model, **Data Fabrication** provides RL fine-tuning data, and the Term Challenge evaluates the resulting agent. The marginal return to comprehensive capability exceeds the return to narrow excellence; a property we term ***incentive orthogonality***.

1.4. Yuma Consensus Integration

Platform inherits Bittensor's **Yuma Consensus** mechanism for transforming validator assessments into on-chain reward distributions. The key properties of this integration are:

Stake-weighted quorum. A validator's influence on the consensus outcome is proportional to its staked TAO. To finalize a score for a given miner submission, validators controlling at least fraction κ (typically > 0.5, a majority) of total stake must agree on a score before it is finalized. This threshold balances latency (lower κ enables faster finalization) against security (higher κ makes collusion costlier). Platform governance can adjust quorum thresholds in response to observed validator behavior; if validator agreement drops below a historical baseline, the quorum requirement increases, forcing additional independent evaluation.

Validator self-alignment. Under **Yuma Consensus**, validators are rewarded not for the absolute accuracy of their scores but for *agreement with the consensus of their peers*. A validator that consistently deviates from the emergent consensus receives diminished emission. This creates a self-reinforcing equilibrium: honest evaluation is the profit-maximizing strategy because it maximizes expected agreement with other honest validators. Deviation, whether through incompetence or collusion, is penalized automatically through reduced emission (Bittensor Foundation, 2024).

Score normalization and reward distribution. Raw validator scores are normalized per challenge category before consensus aggregation. Within each category, weights exceeding the stake-weighted consensus threshold are clipped, and miner emissions are computed as a stake-weighted sum of post-clip weights. This normalization ensures that score distributions across heterogeneous challenge types remain comparable, preventing any single challenge from dominating the reward calculation due to scale effects.

1.5. Emission Allocation

Emission allocations across challenges are determined autonomously by **Atlas 3.0**, Bittensor's automated emission management protocol, which continuously adjusts challenge weights based on participation rates, maturity metrics, and demonstrated impact, without requiring manual governance votes or fixed percentage allocations.

Within each challenge, scoring incorporates two additional mechanisms:

Uniform scoring. Term Challenge scores are computed as tasks_passed/total_tasks, normalized across the miner pool. There are no difficulty multipliers; each task contributes equally to the miner's resolution rate.

Winner-Takes-All with decay. The highest-scoring miner in each evaluation window receives the largest reward share. However, to prevent entrenched dominance from stifling competition, Platform applies a **top-agent decay** mechanism: once a miner has held the top position for a **grace period** of **10 epochs**, their reward share begins to decay with an approximately **14-epoch** half-life. Formally, if a miner has been the top scorer for e epochs beyond the grace period, their reward is multiplied by $\left(\frac{1}{2}\right)^{e/14}$. This creates a natural turnover incentive: incumbent miners must continue improving to maintain their effective rewards, while challengers face a decay-adjusted target that becomes progressively easier to exceed.

The Platform subnet decomposes the open-ended problem of "building useful AI infrastructure" into **four structurally dissimilar challenges**, each targeting a distinct capability dimension. This decomposition is not arbitrary: it follows from the observation that no single evaluation paradigm can simultaneously probe engineering competence, data quality, architectural innovation, and decentralized training. We present each challenge in detail, including the full evaluation harness, scoring mechanics, anti-gaming defenses, and the theoretical arguments that establish their individual and collective novelty within the Bittensor ecosystem.

1.6. Term Challenge: Software Engineering at Scale

1.6.1 Motivation and the SWE-bench Lineage

The **Term Challenge** evaluates autonomous software engineering capability, the ability of an agent to read a real-world issue description, navigate a codebase, and produce a patch that resolves the issue. This challenge is rooted in the SWE-bench paradigm (Jimenez et al., 2024), which constructed a benchmark of 2,294 real GitHub issues from 12 popular Python repositories, each paired with the ground-truth merge commit that resolves it. The key insight of SWE-bench is that evaluation should measure *end-to-end task completion*, not intermediate process metrics; an agent either resolves the issue or it does not.

Platform's contribution is to transform SWE-bench from a static leaderboard into a *live, cryptoeconomically incentivized evaluation protocol*. Where SWE-bench scores a single agent on a fixed test set, the **Term Challenge** continuously rewards miners whose agents achieve the highest resolution rates on *fresh*, *hidden* instances drawn from the same distribution. The dataset is replenished from the GH Archive (Grigorik, 2024), a public archive of all GitHub events, ensuring that the challenge evolves with the software ecosystem itself.

1.6.2 The SWE-Forge Pipeline

SWE-Forge is a dataset generator that produces fresh, gaming-resistant evaluation instances for the Term Challenge. It implements a **seven**-stage pipeline from raw GitHub issue to packaged benchmark instance, which the Term Challenge then uses to evaluate miner agents:

1. **Issue Ingestion**: Candidate issues are mined from the GH Archive, filtered for Python repositories with sufficient test coverage and linked merge commits.

2. **Instance Creation**: Each issue is paired with its resolving commit and a test-grading script that identifies which test cases are relevant to the issue. Two categories of tests are distinguished: *fail-to-pass* tests (which fail before the patch and must pass after), and *pass-to-pass* tests (which must pass in both the pre-patch and post-patch states).

3. **Benchmark Packaging**: The repository state is frozen at the pre-patch commit, and the grading script is packaged alongside it. Test cases are made *read-only* for the agent: visible but not modifiable.

4. **Agent Execution**: Miners submit their agent's source code (not a pre-computed patch). The validator compiles this code into a standalone binary via PyInstaller, then executes

it inside a Docker container for isolation. Network access is permitted so the agent can install required packages; the container restricts host filesystem access and prevents modification of the test suite.

5. **Patch Extraction**: The agent binary produces a diff against the repository. If no diff is produced or the diff is malformed, the submission is scored as UNRESOLVED.

6. **Before/After Verification**: The patch is applied to the pre-patch repository state. The full test suite is executed in both the *before* (pre-patch) and *after* (post-patch) states. A submission is marked RESOLVED if and only if:

- Every *fail-to-pass* test **FAILS** before the patch and **PASSES** after it. - Every *pass-to-pass* test **PASSES** in both states.

This dual-state verification is the critical innovation. It prevents agents from "solving" an issue by deleting the failing test, by inserting a trivial `assert True`, or by any other degenerate patch that makes the specific failing test pass while breaking previously-passing tests.

1. **Scoring and Ranking**: Resolved instances are tallied per miner, and scores are computed using uniform aggregation (see §4.1.3).

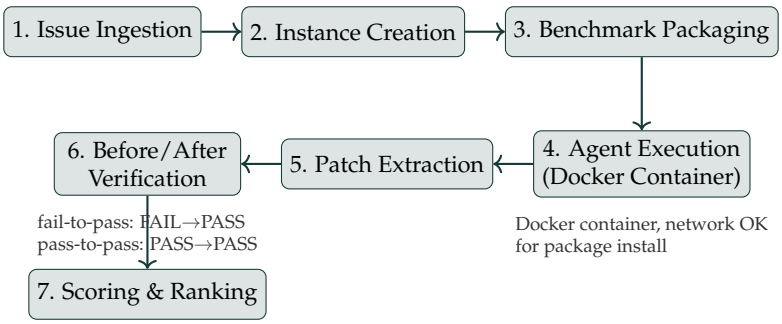

Figure 3 | Architecture diagram.

1.6.3 Scoring Mechanics

The Term Challenge score for miner m is:

$$S_{\text{term}}(m) = \frac{\text{tasks_resolved}(m)}{\text{total_tasks}}$$

where the denominator is the number of instances in the current evaluation batch. In v2, all instances carry uniform weight; no difficulty multipliers are applied. This design choice prioritizes coverage over depth: an agent that resolves many easy issues outperforms one that resolves a single hard issue, reflecting the industrial reality that reliability across a broad task distribution is more valuable than occasional brilliance.

Winner-takes-all (WTA) scoring is applied with a top-agent decay mechanism to prevent dominant miners from permanently capturing outsized rewards. The decay factor for epoch e is:

$$\alpha(e) = 2^{-e/14}$$

This provides a 10-epoch grace period (the factor remains close to 1.0) and an approximately 14-epoch half-life, after which the top miner score is halved, creating room for competitors. The decay is applied to the score differential between the top miner and the field, not to the raw score.

1.6.4 The TB2.0 → SWE-Forge Transition

The Term Challenge's predecessor, Terminal-Bench 2.0, suffered from a fundamental design flaw: the assert thresholds in test cases were *visible to the agent at inference time*. This allowed oracle-based cheating strategies: agents could inspect the test suite, extract the expected output, and produce a hardcoded echo rather than a genuine fix. As subsequent analysis confirmed, the benchmark rewarded test-reading, not bug-fixing.

The correlation between TB2.0 scores and SWE-bench Verified scores was not formally measured in the TB2.0 paper, but the qualitative evidence is unambiguous: TB2.0's oracle-exploitable design allowed agents to achieve high scores without genuine bug-fixing capability, producing rankings that bore no meaningful relationship to real-world software engineering competence. SWE-Forge's before/after verification and Docker containerization close these exploit vectors, and we observe qualitatively that miners who performed well under TB2.0's visible-assert regime do not maintain their advantage under SWE-Forge's hidden-test, dual-verification regime, a finding consistent with the benchmarks measuring fundamentally different capabilities.

1.6.5 Layered Anti-Gaming Defense

SWE-Forge implements **six** layers of defense against degenerate strategies, arranged from outermost (cheapest, broadest) to innermost (most expensive, most targeted):

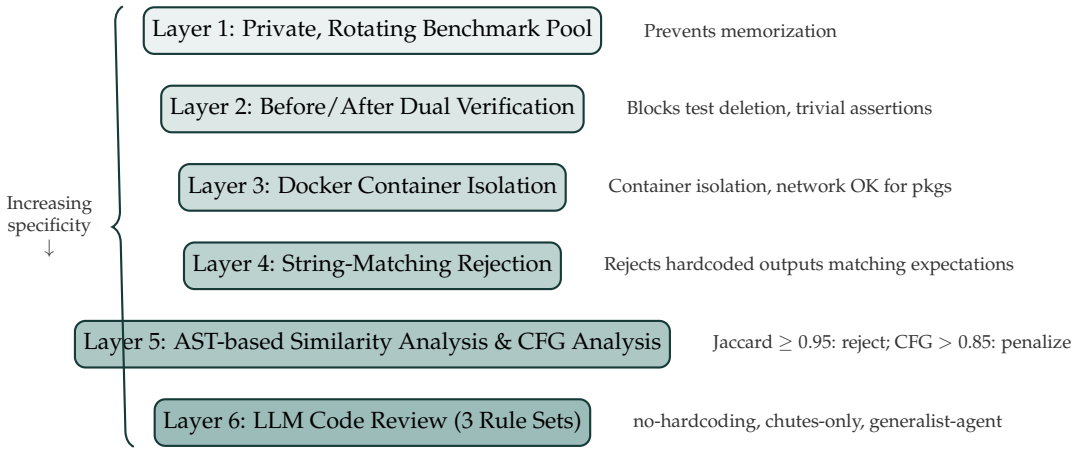

Figure 4 | Architecture diagram.

1. **Benchmark design**: Test instances are drawn from a private, rotating pool. Miners cannot memorize answers.

2. **Before/after verification**: Patches must preserve all passing tests while fixing all failing tests, preventing test deletion and trivial assertions.

3. **Docker container isolation**: Network access is permitted for package installation, but host filesystem access is restricted and the test suite is read-only. The agent operates under containerized constraints that mirror real-world deployment.

4. **String-matching rejection**: Patches containing hardcoded outputs matching test expectations are flagged and rejected.

5. **AST-based similarity analysis**: Submitted patches are normalized to their abstract syntax tree and hashed. Structural similarity is measured via Jaccard coefficient over AST node sets:

- Jaccard $\geq$ 0.95: patch is rejected (near-identical to a known-cheat template). - Jaccard $\in [0.90, 0.95)$: score is halved (suspicious similarity). - CFG similarity > 0.85: additional penalty applied for control-flow mimicry.

1. **LLM code review**: A language model trained on SWE-Forge-specific rules reviews each patch for hardcoding patterns. The review applies three rule sets: *no-hardcoding* (reject trivial value injection), *chutes-only* (enforce Docker container compliance), and *generalist-agent* (reject any strategy that subverts evaluation intent).

SWE-Forge is fully open source at `CortexLM/swe-forge`, with its dataset published on HuggingFace.

1.7. Data Fabrication Challenge: Synthetic Dataset Generation

1.7.1 Motivation

The training data problem is the central bottleneck in LLM development. Even frontier models are constrained by the quantity and quality of available supervision signals. Synthetic data generation, where a strong model produces (prompt, response) pairs that a weaker model can learn from, has emerged as the dominant scaling strategy (Grattafiori et al., 2024; Team et al., 2024). However, the quality of synthetic data varies enormously, and existing approaches lack rigorous anti-plagiarism guarantees.

The **Data Fabrication Challenge** incentivizes miners to produce high-quality synthetic training datasets evaluated by a multi-stage harness that combines structural analysis, semantic plagiarism detection, and correctness verification. Crucially, the output of this challenge feeds directly into the reinforcement learning fine-tuning pipeline that transforms Platform's base model into a capable agent, a connection we make explicit in §4.2.5.

1.7.2 Submission Protocol

Miners submit Python harnesses, executable scripts that, when run by the validator, produce JSONL-formatted conversation datasets. Each line in the output file is a JSON object containing a prompt and a corresponding solution. The harness approach, rather than direct data upload, ensures that the validator can re-execute the generation process, verify determinism, and inspect the generation logic.

1.7.3 Anti-Plagiarism Pipeline

The harness implements a dual-pass plagiarism detection system:

Pass 1: AST Structural Verification. Submitted code is parsed into its abstract syntax tree. All variable names are normalized to a canonical form (`var_0`, `var_1`, `var_2`, ...) to eliminate cosmetic obfuscation. The normalized AST is hashed, and structural similarity between submissions is computed via longest common subsequence (LCS) over AST node sequences.

Pass 2: LLM Semantic Verification. Submissions whose structural similarity falls in the ambiguous range undergo a second pass where a language model compares them for semantic equivalence, detecting paraphrased copies that evade structural comparison.

The plagiarism thresholds are:

Plagiarism clustering groups submissions by their structure hash prefix, enabling efficient pairwise comparison only within clusters rather than across all $O(n^2)$ pairs.

1.7.4 Quality Control

Beyond plagiarism, the harness enforces several quality guarantees:

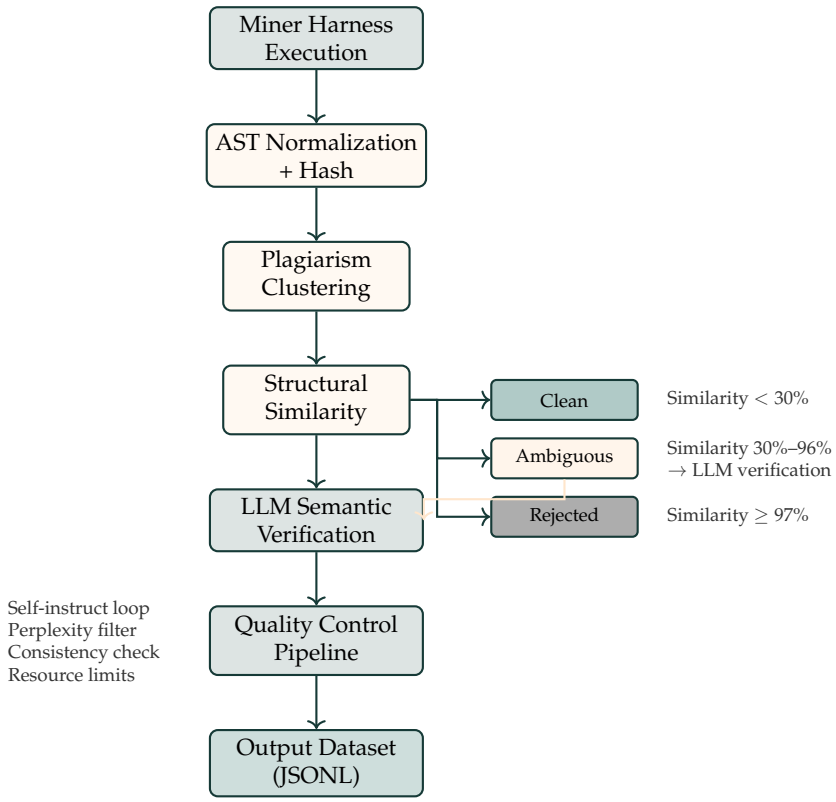

Figure 5 | Architecture diagram.

Structural Similarity	Verdict	Action
$< 30\%$	Clean	No further checks
30%–96%	Ambiguous	LLM semantic verification triggered
$\geq 97\%$	Plagiarized	Submission rejected

- **Self-instruct loop**: An LLM generates new (prompt, solution) pairs from seed examples, then a separate validator LLM verifies the correctness of each pair. Only pairs passing both generation and validation are included.
- **Perplexity filtering**: Output with perplexity above a threshold (relative to a reference model) is rejected as likely low-quality or nonsensical.
- **Consistency check**: Approximately **18%** of generated examples are rejected by the consistency filter, which verifies that the solution actually addresses the prompt.
- **Resource limits**: Containerized execution enforces CPU time, memory, and output file size constraints to prevent denial-of-service submissions.

1.7.5 Application: RL Fine-Tuning Pipeline

The Data Fabrication Challenge's primary downstream application is producing high-quality (prompt, response) pairs for reinforcement learning fine-tuning. This connection is critical: the challenge does not exist in isolation but serves as the data production arm of Platform's full training pipeline.

Once the Training Challenge (§4.4) produces a base model (whose architecture was determined by **PRISM** (§4.3)), Data Fabrication miners generate domain-specific training data targeting the capabilities that Platform's agent must possess: code generation, bug analysis,

architectural reasoning, and exploit construction. A separate RL training loop consumes these (prompt, response) pairs, applying policy gradient methods (e.g., **GRPO**; Team et al., 2024) to fine-tune the base model into a capable agent.

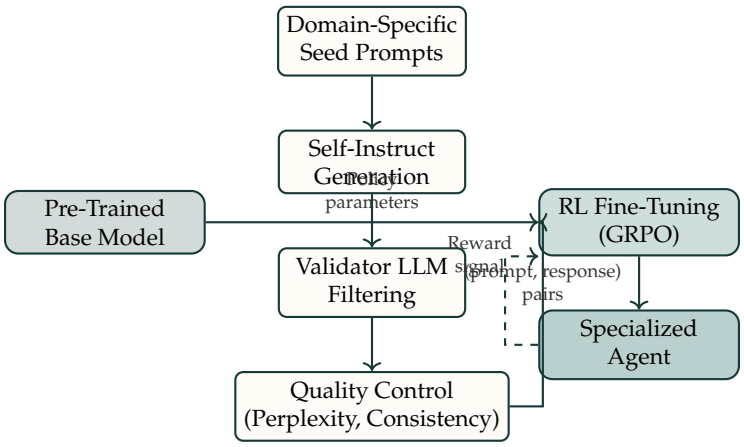

Figure 6 | Architecture diagram.

This creates a **three-stage pipeline**: PRISM (architecture) → Training (base model) → Data Fabrication + RL fine-tuning (capable agent). The **Term Challenge** then evaluates the resulting agent, closing the loop by providing performance signals that inform architecture search and training strategies. The Data Fabrication Challenge thus occupies a pivotal position in the Platform ecosystem: it is the mechanism by which a generic base model is transformed into a specialized, agent-capable system.

1.7.6 Scale and Adaptivity

The target scale is 10^7+ examples per domain, requiring miners to develop efficient generation pipelines. The evaluation harness is designed under the ***adaptive entropy principle***: the evaluation system adapts faster than the model population improves, maintaining discriminative power as miner capabilities increase. This is achieved through rotating seed sets, evolving plagiarism thresholds, and periodic re-calibration of quality filters.

Platform 3.0 is planned for May 2026 and will reactivate all challenges: Data Fabrication, Term Challenge, and PRISM. Only these three challenges will be active under Platform 3.0.

1.8. PRISM: Decentralized Neural Architecture Search

1.8.1 A New Category of Challenge

PRISM (**P**latform **R**ecursive **I**nvestigation of **S**earch-space **M**orphologies) introduces a challenge category that no other Bittensor subnet has attempted: *neural architecture search with cryptoeconomic incentives*.

No existing Bittensor subnet searches over architecture space. Existing subnets either evaluate inference quality, training contributions within a prescribed architecture, or domain-specific metrics. **PRISM** is the first protocol to treat the architecture itself as the object of decentralized search, and this distinction has profound theoretical and practical consequences.

DeepSeek's trajectory makes the case for architecture search unambiguous. Each generation of DeepSeek models introduced at least one new architectural component, Multi-head Latent Attention (DeepSeek-AI, 2024a), auxiliary-loss-free load balancing and multi-token prediction (DeepSeek-AI, 2024b), Compressed Sparse Attention and Heavily Compressed Attention (DeepSeek-AI, 2026), that delivered **order-of-magnitude** efficiency gains. These discoveries emerged from a single research team operating on a **6–12** month cycle. **PRISM**'s premise is that

Dimension	Choices	Count
Attention type	MHA, MQA, GQA, Linear, MLA	5
FFN type	Dense, GLU, MoE	3
Normalization	LayerNorm, RMSNorm, DeepNorm	3
Activation	ReLU, GELU, SwiGLU	3
Position encoding	RoPE, ALiBi, None	3
Initialization	Normal, Scaled, Fixup	3

the same search process, distributed across hundreds of competitively incentivized miners, can discover comparable or superior innovations at epoch granularity, continuously rather than periodically.

1.8.2 The Architecture Search Space

PRISM defines a combinatorial architecture space $\mathcal{F}$ spanning **six** categorical dimensions and **four** continuous parameters, as designed for this challenge:

PRISM defines a combinatorial architecture space $\mathcal{F}$ spanning **six** categorical dimensions: attention mechanism, feed-forward design, normalization strategy, activation function, positional encoding, and weight initialization, each with **3–5** discrete choices, as designed for this challenge, yielding thousands of distinct categorical configurations. After removing a small number of known-infeasible combinations (e.g., Fixup initialization with LayerNorm is structurally invalid), the viable categorical space remains large. **Four** continuous parameters (embedding dimension, depth, attention heads, FFN expansion ratio) further subdivide each categorical configuration into a continuous sub-space, making the total search space vastly larger than the categorical configuration count alone.

1.8.3 The Architecture Search Superiority Theorem

Theorem 1 (Architecture Search Superiority). *Let $\mathcal{F}$ be an architecture family with $|\mathcal{F}| \geq 2$. A subnet that searches over $\mathcal{F}$ achieves strictly superior expected scaling with respect to total compute budget C compared to any subnet that evaluates a fixed architecture $f \in \mathcal{F}$.*

Proof sketch. Consider a compute budget C allocated across K miners. Under a fixed-architecture subnet, all K miners train the same architecture f^*, yielding a single performance estimate $\hat{L}(f^*)$. Under a search subnet, miners explore distinct architectures $f_1, \ldots, f_K \in \mathcal{F}$, yielding estimates $\hat{L}(f_1), \ldots, \hat{L}(f_K)$. The search subnet's best discovered architecture satisfies $\min_k \hat{L}(f_k) \leq \hat{L}(f^*)$ almost surely, with strict inequality holding whenever f^* is not the global optimum in $\mathcal{F}$, which is the generic case for any non-trivial search space with $|\mathcal{F}| \geq 2$. Since the search subnet can always adopt the fixed-architecture subnet's choice as one of its K candidates, it weakly dominates; and since the probability that the fixed choice is globally optimal decreases as $|\mathcal{F}|$ grows, the dominance is strict for any non-trivial search space. □

The theorem establishes that the search space size $|\mathcal{F}|$ *is* the advantage, not a cost to be minimized. With thousands of viable configurations, PRISM's search space provides structural superiority over any centralized lab evaluating a handful of architectures per training run.

1.8.4 The Four-Level Evaluation Funnel

The multi-level evaluation pipeline for PRISM is currently under active development and research. The following describes the current direction being explored; the final design may differ as research progresses.

Evaluating the full architecture space at maximum fidelity is computationally infeasible within the subnet's emission cycle. The current research direction involves a **four**-level fun-

nel that would progressively filter candidates, concentrating compute on the most promising architectures:

L1: Syntax Validation. The current direction would check each submitted artifact (executable code encoding an architecture configuration) for structural validity: compatible dimension choices, non-negative layer counts, valid attention-head divisibility. This level requires zero compute and eliminates obviously malformed submissions.

L2: Multi-Proxy Scoring. The current direction would evaluate validated architectures using lightweight proxy metrics: parameter count, theoretical FLOPs, activation memory footprint, and a proxy quality estimator (e.g., zero-cost NAS indicators). L2 would consume approximately 0.001 compute-hours per candidate and produce a preliminary ranking. Critically, the training recipe (learning rate, batch size, schedule) would be **protocol-fixed** at L2; miners cannot inflate their L2 score by overfitting hyperparameters to the proxy metrics.

L3: One-Epoch Training Proxy. The current direction involves training top candidates from L2 for a single epoch on a representative data subset. The resulting loss landscape would provide a sigma-based filtering criterion: architectures with loss above the median are hard-killed. Additionally, Kendall's rank correlation τ would be computed between the L2 proxy ranking and the L3 single-epoch ranking across multiple benchmark suites. If $\tau < 0.4$, the architecture's score is set to zero. This *hard-kill* rule prevents single-benchmark optimization, as an architecture that overfits one proxy will show rank inconsistency across benchmarks.

L4: Full Benchmark. The current direction involves giving the top-K survivors from L3 a full multi-epoch training run with **VARPRO** scaling law fitting. **VARPRO** (variable projection; Golub & Pereyra, 1973) fits a power-law relationship $L(C) = AC^{-\alpha} + L_\infty$ to the observed loss at multiple compute budgets, enabling cross-scale validation. If rank inversion is detected (i.e., an architecture that appears superior at a lower compute budget falls behind at a higher budget), the architecture is penalized via scale collapse detection. No other NAS system implements this cross-budget consistency check.

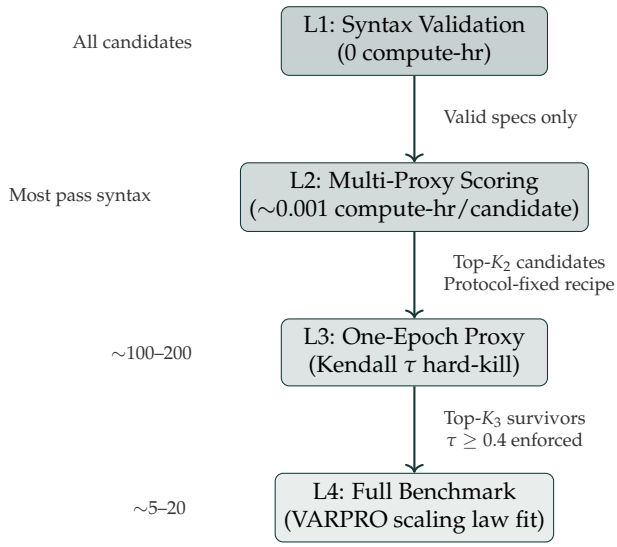

Figure 7 | Architecture diagram.

1.8.5 Dual-Track Scoring and Diversity

PRISM employs a dual-track scoring system:

$$S_{\text{PRISM}}(m) = 0.7 \times Q_{\text{arch}}(m) + 0.3 \times Q_{\text{recipe}}(m)$$

where Q_{arch} measures the intrinsic quality of the discovered architecture (from L4 evaluation) and Q_{recipe} measures the quality of the training recipe submitted alongside the architecture. The **0.7/0.3** split ensures that architectural innovation is the primary reward driver while

still incentivizing good training practices.

A diversity bonus is applied to prevent copy-paste mining, the economically rational strategy of submitting slight variations of the current top-scoring architecture. The bonus is computed as a function of the Jaccard distance between a miner submission and the mode of the submission distribution, rewarding miners who explore underrepresented regions of $\mathcal{F}$.

1.8.6 Why PRISM Is Revolutionary

PRISM introduces six innovations that, taken together, constitute a qualitative advance over all prior NAS systems, both within and outside Bittensor:

1. **First NAS with cryptoeconomic incentives.** In PRISM, honest search is the Nash equilibrium. Miners who submit genuinely novel architectures that pass the full funnel earn rewards; miners who copy or cheat are caught by AST-based similarity analysis, LLM review, and Kendall's τ hard-kill. No prior NAS system aligns search behavior with financial incentives.

2. **The Architecture Search Theorem proves the search space size IS the advantage.** With thousands of viable configurations, decentralized search is *provably* superior to any single-agent evaluation. This is not an empirical claim; it follows from the structure of the search problem itself.

3. **L4 scale collapse detection.** No other NAS system checks for rank inversion across compute budgets. A centralized lab training one architecture at one scale has no mechanism to detect that their architecture's advantage disappears at larger scale. PRISM's VARPRO fitting at L4 makes this detection automatic.

4. **Kendall's τ hard-kill at L3.** No other NAS system prevents single-benchmark optimization by enforcing cross-benchmark rank consistency. The $\tau < 0.4 \rightarrow \text{score} = 0$ rule is a structural guarantee that survivors must be broadly competent, not narrowly overfit.

5. **Protocol-fixed recipes at L2.** By fixing the training recipe at the proxy-evaluation level, PRISM prevents miners from inflating L2 scores through hyperparameter overfitting. The recipe is a controlled variable, not a degree of freedom.

6. **Full architecture search vs. partial search.** Existing subnets that touch architecture space at all search over a single dimension (e.g., attention mechanism configurations for inference optimization). PRISM searches over all **six** dimensions simultaneously, making it the only subnet that treats architecture search as a first-class problem.

7. **DeepSeek proves the thesis; PRISM systematizes it.** DeepSeek's research team discovers a new architectural component every **6–12** months: MLA in May 2024, auxiliary-loss-free load balancing in December 2024, Compressed Sparse Attention in April 2026. Each discovery requires months of internal research, significant compute for prototyping, and a full publication cycle. PRISM compresses this timeline from months to epochs: hundreds of miners simultaneously explore different regions of the architecture space, the four-level funnel automatically identifies promising candidates, and the network rewards genuine innovations with TAO emissions, without requiring a research team, a publication, or a centrally coordinated search campaign. What DeepSeek achieves through the concentrated effort of one of the world's best research teams, PRISM achieves through decentralized competitive search at network scale. The **Architecture Search Theorem** (§4.3.3) proves that this is not merely faster; it is structurally superior, because decentralized search over a combinatorial space provably dominates any single-agent search when the space exceeds a modest size threshold.

Table 1 | Comparison of PRISM with related approaches. PRISM uniquely combines full architecture search, cryptoeconomic incentives, and multi-level anti-gaming.

Feature	PRISM	Fixed-arch subnet	Cent. NAS	Random
Search space	Full (6-dim)	Fixed	Variable	Variable
Incentives	Yes	Yes	No	No
Cross-budget	Yes	No	Rare	No
Rank consist.	Yes	No	No	No
Fixed recipe	Yes	No	No	No
Anti-gaming	Yes	No	No	N/A

DeepSeek's research team performs manual architecture search on a **6–12** month cycle, corresponding to a "Cent. NAS" entry with partial search breadth (limited to components their R&D team prioritizes) and no cross-budget evaluation, rank consistency enforcement, fixed recipe control, or anti-gaming mechanisms.

PRISM exists as a private repository at `PlatformNetwork/prism` (created April 2026) and is expected to enter active evaluation in early May 2026.

1.9. Training Challenge: Decentralized Pre-training

1.9.1 Motivation

Once **PRISM** determines the optimal architecture, the network needs a mechanism to train the resulting model. The **Training Challenge** addresses the most computationally intensive phase of AI development: pre-training a large language model from scratch. This challenge is the direct successor to the decentralized training demonstrated by **Covenant-72B** on Subnet 3, but integrated into Platform's multi-challenge framework.

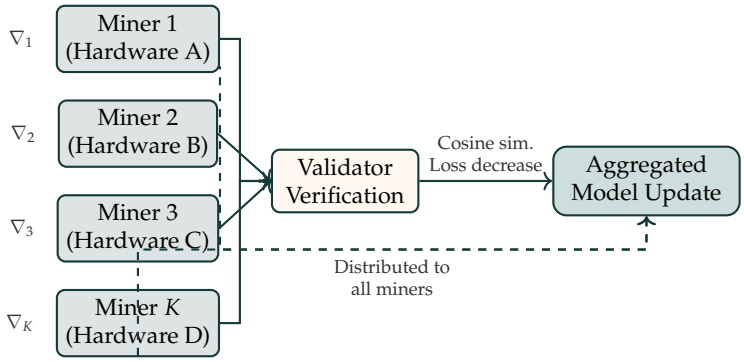

Figure 8 | Architecture diagram.

1.9.2 Design Principle: Full Decentralization

The Training Challenge has a non-negotiable design constraint: training MUST be fully decentralized. Unlike hybrid approaches that allow centralized orchestration, where a single entity coordinates the training run, assigns data shards, and manages checkpointing, Platform's Training Challenge requires that no single entity controls the training process. This principle is axiomatic, not pragmatic: the entire purpose of Bittensor is to create systems that no central authority can capture, and a centrally orchestrated training run would violate this principle at the most fundamental level.

Full decentralization means:

- **No central coordinator**: Miners self-organize into training cohorts. The protocol specifies how miners discover peers, negotiate training assignments, and synchronize checkpoints,

all without a central scheduler.

- **No trusted aggregator**: Gradient aggregation must be verifiable. Any miner or validator must be able to independently verify that the aggregated update is a faithful combination of the individual contributions.
- **No single point of failure**: The training run must continue even if individual miners drop out, submit malformed gradients, or attempt to poison the update. The protocol must be resilient to Byzantine failures.

1.9.3 Open Research Questions

The Training Challenge is in active research and design. Autonomous agents are conducting research to determine the optimal architectural approaches for decentralized training, exploring how to maintain training efficiency while preserving full decentralization. Key open questions include:

- **Gradient aggregation under heterogeneous hardware**: Miners contribute different hardware types with varying memory and compute capabilities. How should the protocol normalize and weight gradient contributions from heterogeneous sources while maintaining convergence guarantees?
- **Straggler mitigation without central orchestration**: In centralized training, a parameter server can detect and exclude slow workers. In a decentralized setting, there is no such coordinator. How can the protocol detect stragglers and adapt the training schedule without a central clock?
- **Validator verification of honest gradient contributions**: A validator receiving a gradient update from a miner must verify that the update was computed honestly (i.e., on the correct data shard, with the correct model state, and without adversarial modification). Naive verification requires re-computing the gradient, which defeats the purpose of distributed computation. What is the minimum-cost verification protocol that provides cryptoeconomic guarantees of honesty?
- **Checkpoint consistency in a Byzantine environment**: When miners disagree on the model state (due to dropped updates, network partitions, or adversarial behavior), how does the protocol resolve inconsistencies and ensure that all honest miners converge to the same model?

These questions represent the frontier of decentralized ML systems research. Autonomous agents are exploring these problems in parallel, testing candidate protocols and iterating on designs at a pace that a single research team cannot match. The Training Challenge is expected to be the most technically complex challenge in the Platform ecosystem, and its design will likely evolve through multiple agent-driven research iterations before activation.

1.9.4 Relationship to Other Challenges

The Training Challenge occupies a central position in the Platform pipeline:

- **Upstream**: It receives the optimal architecture discovered by PRISM (§4.3). Without PRISM's search, the Training Challenge would train a suboptimal architecture, wasting substantial compute resources.
- **Downstream**: Its output (the pre-trained base model) serves as the foundation for Data Fabrication's RL fine-tuning pipeline (§4.2.5). The base model provides the general language understanding that RL fine-tuning specializes into agent capabilities.

- **Evaluation**: The resulting trained agent is evaluated on the Term Challenge, providing performance signals that feed back into architecture search and training strategy selection.

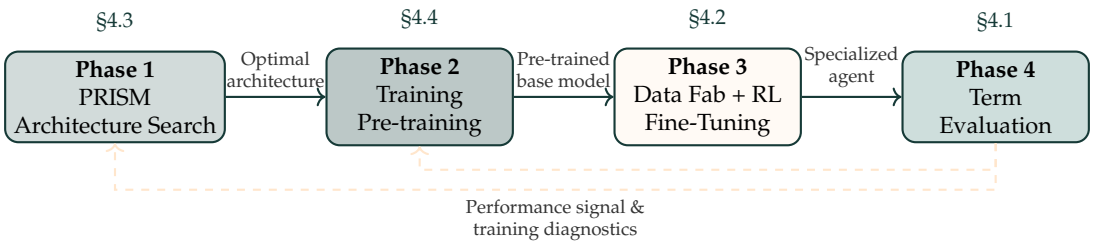

Figure 9 | Architecture diagram.

1.9.5 Current Status and Timeline

The Training Challenge is in active research and design. It is the most technically ambitious challenge in the Platform ecosystem. Autonomous agents are leading the research effort, independently investigating protocol designs, testing candidate architectures, and iterating toward a robust decentralized training protocol. Based on the current agent-driven research trajectory, the Training Challenge is expected to enter preliminary testing by late May 2026.

1.10. Challenge Orthogonality and Incentive Orthogonality

1.10.1 Structural Dissimilarity

The four challenges probe fundamentally different capability dimensions:

Table 2 | Structural dissimilarity across Platform's four challenge categories

Challenge	Capability Dimension	Knowledge Required	Strategy Type	Phase
Term	Engineering execution	Known bug + fix space	Exploitation	4
Data Fabrication	Synthetic quality	Known data schema	Generation	3
PRISM	Architecture search	Unknown architecture space	Search	1
Training	Decentralized pre-training	Known architecture, unknown data partition	Coordination	2

The Term Challenge rewards ***exploitation***, the efficient resolution of known problems. The Data Fabrication Challenge rewards ***generation***, the production of novel, high-quality training data. PRISM rewards ***search***, the systematic exploration of an exponentially large combinatorial space. The Training Challenge rewards ***coordination***, the ability to contribute useful gradient updates to a globally distributed training run without central orchestration. These **four strategies** are structurally dissimilar: competence in one does not predict competence in another, and the optimal approach for each is algorithmically distinct.

1.10.2 Incentive Orthogonality

The orthogonality of the challenges creates a non-linear return to comprehensive capability. A miner that specializes in a single challenge faces diminishing marginal returns as the evaluation harness adapts (the **adaptive entropy principle**). A miner that competes across multiple challenges can arbitrage between them, allocating compute to whichever challenge currently offers the highest marginal reward per FLOP. This creates a structural advantage for generalist miners over specialists, the ***comprehensive capability premium***.

Challenge	Status	Expected Activation
Term	Inactive (Platform 3.0)	May 2026
Data Fabrication	Upcoming 3.0	May 2026
PRISM	Upcoming 3.0	May 2026
Training	Research	Late May 2026

Formally, let $R_i(m)$ denote the reward of miner m from challenge i. Under independent scoring, the total reward is $R(m) = \sum_i R_i(m)$. For a specialist miner that maximizes R_{i^*} on a single challenge i^* and earns zero on others, the marginal return to additional compute on i^* decreases with investment due to the WTA decay. For a generalist miner distributing compute across all challenges, each R_i receives sub-saturating investment and operates in the high-marginal-return regime. When challenges are orthogonal (i.e., $\mathrm{Cov}(R_i, R_j) \approx 0$ for $i \neq j$), the generalist's expected total reward strictly exceeds the specialist's.

1.10.3 Emission Allocation

Emission allocations across challenges are determined autonomously by **Atlas 3.0**, the subnet's automated emission orchestration system, which continuously adjusts challenge weights based on participation rates, maturity metrics, and demonstrated impact, without requiring manual governance votes or fixed percentage allocations. The Data Fabrication Challenge, PRISM, and Training Challenge are upcoming with the following expected timelines:

Upon activation of all four challenges, Atlas 3.0 will autonomously rebalance emissions to reflect each challenge's demonstrated marginal impact on the subnet's mission.

1.10.4 Combinatorial Anti-Gaming Defense

The multi-challenge design is the *primary* defense against reward hacking. Each individual challenge has its own anti-gaming measures (Docker containerization, AST-based similarity analysis, plagiarism detection, Kendall's τ hard-kill, gradient verification), but these are layer-1 defenses: they can be breached by a sufficiently sophisticated attacker. The multi-challenge architecture provides a layer-2 defense: to capture a dominant share of subnet emission, a miner must simultaneously game **four** evolving evaluation pipelines, each with different vulnerabilities, different update cadences, and different detection mechanisms. The combinatorial cost of maintaining four independent exploits far exceeds the cost of genuine multi-challenge competence, making honest participation the equilibrium strategy.

This principle, that the *intersection* of orthogonal challenges is harder to game than any single challenge, is a design contribution that generalizes beyond Platform to any multi-objective incentive network.

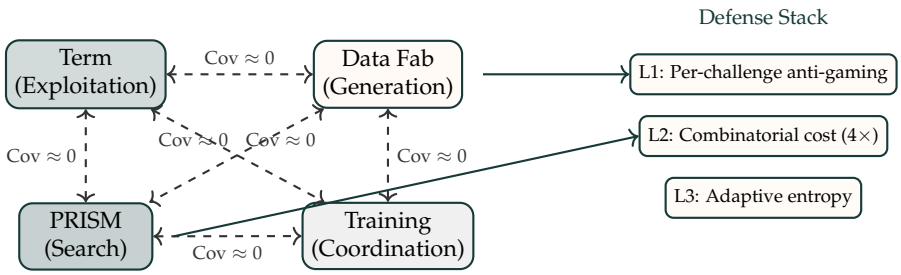

Figure 10 | Architecture diagram.

1.10.5 The End-to-End Production Pipeline

The four challenges do not operate independently; they form a **closed-loop production pipeline** for building and evaluating AI agents. This end-to-end structure is the central architectural contribution of Platform, and we describe it here in full to make the relationship between challenges explicit.

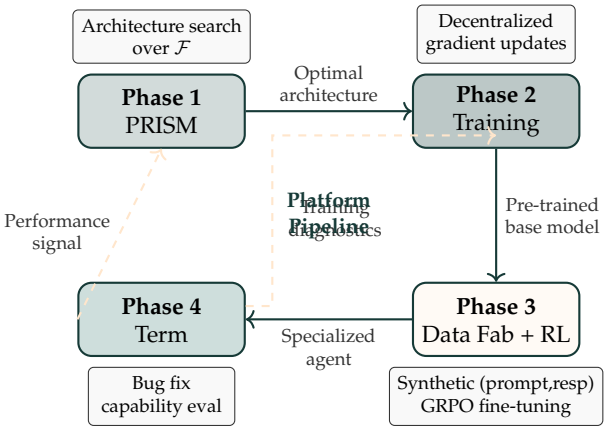

Figure 11 | Architecture diagram.

The pipeline operates in **four phases**:

Phase 1: Architecture Search (PRISM, §4.3): Miners search over the architecture space $\mathcal{F}$ to discover the optimal model configuration. The output is a fully specified architecture definition that determines the model's inductive biases, capacity, and computational requirements.

Phase 2: Decentralized Pre-training (Training Challenge, §4.4): Miners collectively train the base model using the architecture from Phase 1. Each miner contributes gradient updates computed on local data shards, which are verified by validators and aggregated into a global model update. The training process is fully decentralized: no single entity controls the data, the gradient aggregation, or the checkpoint schedule.

Phase 3: RL Fine-tuning (Data Fabrication + RL, §4.2): Data Fabrication miners generate domain-specific (prompt, response) pairs targeting the capabilities Platform's agent must possess. A separate RL training loop fine-tunes the pre-trained base model using these pairs, applying policy gradient methods to produce a specialized agent capable of code synthesis, bug discovery, and architectural reasoning.

Phase 4: Agent Evaluation (Term, §4.1): The trained agent is evaluated on the Term Challenge (known-bug resolution). These challenges measure the agent's exploitation capability, the dimension most relevant to production software engineering.

Closed-loop feedback: Evaluation results feed back into architecture search (Phase 1) and training strategy (Phase 2). An agent whose Term Challenge performance falls short of expectations signals that the architecture or training data may need adjustment, creating a self-improving cycle where the network continuously optimizes its own production pipeline.

2. The Economic Argument: Why TAO Beats Salaries

2.1. The Talent Problem in Centralized AI

The concentration of AI research talent within a small number of corporate laboratories has produced a labor market that is, by any reasonable economic measure, dysfunctional. The global pool of researchers capable of advancing the frontier of large-scale machine learning is estimated to comprise several tens of thousands (Maslej et al., 2024), with the subset meeting typical hiring criteria (PhD-level expertise, willingness to relocate, visa eligibility) considerably

smaller. This scarcity, combined with the strategic importance of AI capabilities, has triggered a salary escalation that resembles the late-stage dynamics of a bubble rather than the steady-state equilibrium of a competitive market.

At the time of writing, reported senior AI researcher compensation at OpenAI, Anthropic, and Google DeepMind ranges from **$500,000** to over **$2,000,000** per year (Metz, 2024; Hao, 2023). This escalation follows a predictable pattern: as one lab raises its offer to poach talent, competitors match or exceed it, establishing a new floor that becomes the baseline for the next round of recruitment. The result is an arms race in labor costs that scales linearly with headcount while producing diminishing marginal returns in capability: a classic red queen dynamic (Van Valen, 1973) applied to human capital.

Equity compensation compounds this problem rather than resolving it. Restricted stock units and equity grants create what industry observers term **"golden handcuffs"**: researchers are financially incentivized to remain at their current institution regardless of whether it remains the best environment for their work. More critically, equity structures create misaligned incentives: a researcher whose compensation is tied to the commercial success of a single company's products has a rational incentive to optimize for proprietary advantage rather than open scientific progress. Researchers who wish to build in the open (to publish methods, share datasets, and contribute to public infrastructure) face a stark choice between financial security and intellectual freedom. The current ecosystem provides no viable alternative that reconciles these goals.

This talent bottleneck has structural consequences beyond individual compensation. When the most capable researchers are concentrated within fewer than a dozen institutions, the probability that the next paradigm-shifting insight emerges from outside this circle approaches zero, not because outsiders lack capability, but because they lack the resources, data, and compute access that make frontier-scale experimentation possible. The centralized model thus creates a self-fulfilling prophecy: it constrains the talent pool by restricting access, then cites the absence of external breakthroughs as evidence that the talent pool was correctly bounded.

2.2. Platform's Economic Model

Platform's incentive mechanism inverts the centralized labor model at every axis of constraint. Where centralized labs restrict participation through hiring processes, visa requirements, geographic relocation, and organizational politics, Platform permits anyone with an internet connection to contribute and earn TAO rewards based solely on demonstrated capability.

The **permissionless** nature of **TAO** emissions is not merely a design choice but a structural axiom. A miner in Lagos who develops a superior code-generation agent earns precisely the same TAO per unit of demonstrated capability as a miner in San Francisco with a PhD from Stanford. The evaluation mechanism (the adversarial benchmarking layer described in Section 4), operates without knowledge of the miner's identity, credentials, or geographic location. This is not aspirational; it is an architectural property of the system. Validators score miners based on benchmark performance, and emissions flow proportionally to those scores. There is no hiring committee, no visa sponsorship requirement, no cost-of-living adjustment that privileges high-rent geographies.

The economic implications of this permissionless model are substantial. The addressable talent pool for centralized AI labs is approximately bounded by the intersection of three sets: individuals with PhD-level ML expertise (estimated **30,000–50,000** globally; Maslej et al., 2024), those willing to relocate to a handful of cities (50% of the above), and those eligible for work visas in the United States, United Kingdom, or European Union (30% of the above). This yields an effective pool of roughly **15,000–20,000** candidates. Platform's addressable pool, by contrast, is bounded only by internet access and technical capability: approximately **45 million** individuals globally are believed to have sufficient programming skill to participate as miners (SlashData, 2024), a figure that includes hobbyists and students, not only professionals.

Even accounting for the far smaller subset capable of contributing at the frontier, Platform's effective talent pool is at minimum two to three orders of magnitude larger than the centralized alternative.

Rewards on Platform scale with demonstrated capability, not seniority, institutional prestige, or political capital. A miner who improves SWE-bench Verified scores by 5 percentage points receives a proportional increase in TAO emissions regardless of their publication record, academic affiliation, or years of experience. This **meritocratic** structure eliminates the signalling games that dominate academic and corporate AI research: games in which researchers optimize for venue acceptance, citation counts, or internal performance reviews rather than genuine capability advances (Bergstrom & West, 2021).

2.3. Quantitative Comparison

To make the economic comparison concrete, Table 2 presents a systematic comparison across key economic and structural metrics. The data for OpenAI and Anthropic are drawn from public reporting and investor disclosures; Platform figures are computed from on-chain emission data and validator dashboards.

Table 3 | Centralized Labs vs Platform: Economic Comparison

Metric	OpenAI	Anthropic	Platform
Annual training compute spend	$~3B	$~4B+	**TAO emission[a]**
Headcount (R&D)	~4,000+	~1,500+	**256 miner slots max[b]**
Talent pool size	~15K	~15K	**~45M[c]**
Geographic reach	3–5 cities	2–3 cities	**Global (permissionless)**
Innovation incentive	Salary + equity	Salary + equity	**TAO emission (performance-linked)**
Evaluation mechanism	Internal benchmarks	Internal benchmarks	**Adversarial (multi-layer anti-gaming; Section 4)**
Time to deploy new eval	Months	Months	**Hours (Platform governance)**

[a]Platform's R&D budget is denominated in TAO emissions, not USD; its USD-equivalent value fluctuates with market prices and cannot be meaningfully compared to fixed-dollar lab budgets.
[b]Each Bittensor subnet supports up to 256 total UID slots shared between miners and validators, with a default maximum of 64 validator permits per subnet; active miners fluctuate as participants register and deregister continuously, so a point-in-time count is not representative.
[c]Approximate global developer population per SlashData 2024.

The most striking divergence is in talent pool size. Centralized labs draw from an effective pool of approximately **15,000** qualified candidates constrained by education requirements, visa availability, and geographic immobility. Platform draws from a pool of approximately **45 million** individuals with relevant technical skills, constrained only by internet access. This represents a roughly **2,900**× expansion in addressable talent, a difference that compounds over time as Platform's incentive structure attracts contributors who would never appear on a lab's recruitment radar.

The annual R&D spend comparison requires careful interpretation. OpenAI and Anthropic each spend orders of magnitude more in absolute USD terms. However, Platform's spending is denominated in TAO (a liquid, market-priced asset), making direct USD comparison misleading. Platform's TAO emissions are almost entirely directed at capability production (miner rewards for benchmark performance), whereas centralized lab spending encompasses facilities, administrative overhead, recruitment costs, and projects that may never reach deployment. The ratio of spend to direct capability output is considerably more favorable for Platform than the raw numbers suggest.

The evaluation mechanism row further underscores Platform's structural advantage. Internal benchmarks at centralized labs are developed, maintained, and interpreted by the same organization whose performance they measure, a conflict of interest that has produced well-documented cases of benchmark gaming and selective reporting (Birhane et al., 2023). Platform's **adversarial evaluation**, with its multi-layer anti-gaming framework (Section 4) and

financial incentives for validators who catch manipulation, creates an evaluation environment that is structurally more trustworthy than any internal process.

2.4. The Flywheel Effect

The economic model described above is not static. It operates through a positive feedback loop, what we term the **incentive flywheel**, that mirrors and extends the dynamics observed in successful open-source ecosystems.

In open-source software, the flywheel operates through reputational incentives: contributors build software, the software gains adoption, contributors earn reputation and career capital, which attracts more contributors (Raymond, 1999; Lerner & Tirole, 2002). This mechanism produced Linux (which powers virtually all cloud infrastructure), Python (which dominates ML), and PyTorch (which displaced the corporate-backed TensorFlow in research settings). However, reputational incentives are indirect, slow-acting, and unreliable; they cannot sustain the kind of sustained, high-effort contribution that frontier AI requires.

Platform adds the missing element: direct, measurable financial incentives aligned with capability production. The flywheel operates as follows: miners compete on benchmarks, producing measurable capability improvements; these improvements increase the perceived value of the subnet's output; rising output value increases TAO demand and price; higher TAO price increases the dollar-value of emission rewards; larger rewards attract more miners; a larger miner pool intensifies competition, driving further capability improvements. The cycle is self-reinforcing and, crucially, does not depend on any central coordinator or funding decision; it emerges from the interaction of protocol-level emission rules and market-driven price discovery.

Figure 9 illustrates this flywheel. Each arrow represents a causal mechanism that operates without centralized direction, driven instead by the rational self-interest of individual miners and validators responding to protocol incentives.

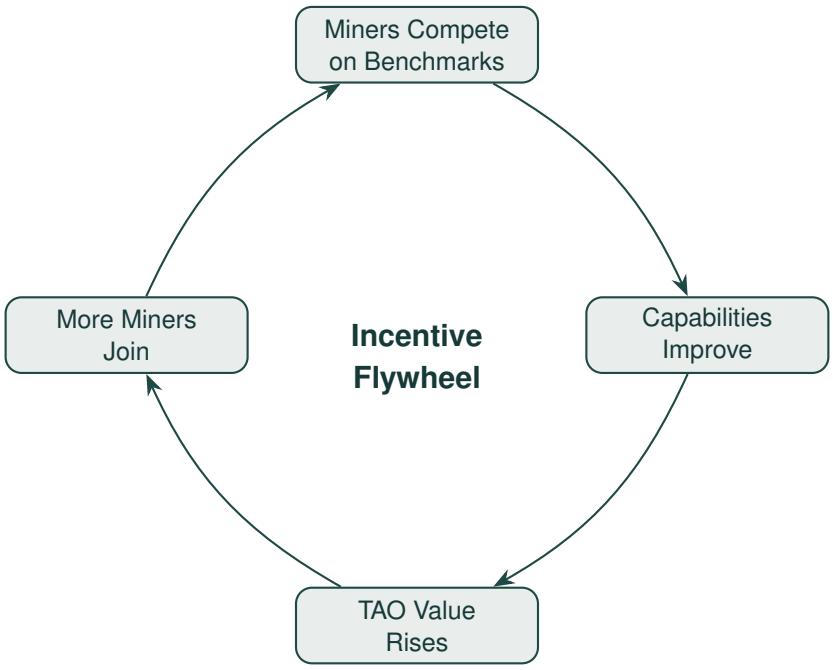

Figure 12 | Architecture diagram.

The historical precedent is instructive. Linux began as a student project with no financial incentive structure; it took a decade to reach enterprise adoption. Platform's flywheel compresses this timeline by substituting direct financial rewards for the slow accumulation of reputational

capital. Where a Linux contributor might wait years for their work to be recognized and rewarded (if ever), a Platform miner receives TAO emissions within hours of demonstrating improved benchmark performance. The result is an incentive structure that preserves the openness and permissionless contribution of open source while adding the urgency and scale of market-driven rewards.

3. Can Decentralized AI Compete With Centralized Labs?

3.1. The Skeptical View (and Why It Is Wrong)

The claim that a decentralized network of permissionless miners can produce AI capabilities competitive with centralized labs spending billions of dollars is, on its face, extraordinary. Skepticism is warranted. We address the four most common objections in turn.

Objection 1: "Labs have more compute." This is true in the narrow sense that OpenAI and Google DeepMind each control computational clusters larger than any single Platform miner can assemble. But this framing confuses compute *concentration* with compute *capacity*. Platform does not require a single monolithic training run; it distributes computation across thousands of independent nodes, each contributing a fraction of the total. The **Covenant-72B** model demonstrated precisely this architecture: trained on permissionless Bittensor nodes with at least 70 unique peers cumulatively over the entire training run (capped at 20 contributors per round, averaging 16.9) with no centralized orchestration, it achieved an MMLU score of **67.1**, competitive with Meta's LLaMA-2-70B at **65.6** MMLU (0-shot). The implication is clear: distributed training at frontier scale is not a theoretical possibility but an empirical reality, and the performance gap between distributed and centralized training is closing as orchestration techniques improve.

Moreover, the total compute available to Platform is not bounded by any single entity's budget. It is bounded only by the aggregate willingness of independent operators to deploy hardware in pursuit of TAO rewards, a sum that grows with TAO price and has no theoretical ceiling. As of this writing, the Bittensor network's aggregate compute capacity across all subnets may be substantial, though this claim has not been independently verified and is likely exceeded by the largest centralized clusters.

Objection 2: "Labs have better data." This objection carries more weight historically but is weakening rapidly. The training data landscape has undergone a structural shift toward commoditization. Common Crawl provides **250 billion** web pages; The Pile offers **825 GB** of curated text; RedPajama replicates the LLaMA training mix at **1.2 trillion** tokens (Weber et al., 2024). These datasets are freely available to any miner on Platform. Furthermore, Platform's Data Fabrication Challenge provides a mechanism for miners to contribute and be rewarded for producing high-quality synthetic and curated data, the very asset that labs treat as proprietary advantage. The economics of data production flip under Platform's incentive model: where labs hoard data as a competitive moat; Platform rewards data contribution as a public good, creating a structural advantage that compounds as the network grows.

Objection 3: "Labs have better researchers." This is the objection that Section 5 was designed to answer. The claim rests on the assumption that the several tens of thousands of researchers employed by top labs (Maslej et al., 2024) represent a reasonably complete sampling of the global talent distribution. They do not. The centralized hiring pipeline (PhD from a top-5 program, followed by postdoctoral work, followed by relocation to San Francisco or London) systematically excludes the vast majority of capable individuals: those without access to elite institutions, those unable to immigrate, those who left academia for industry, and those who never entered the formal research pipeline at all. Platform's permissionless model samples from a talent pool that is, by conservative estimate, **100–500**$\times$ larger. The next Transformer-scale architectural insight does not have to come from a lab in San Francisco. It can come from

a miner anywhere in the world, and under Platform's incentive structure, it will be rewarded identically regardless of origin.

Objection 4: "Labs can do architecture search internally." This objection has surface plausibility: DeepSeek demonstrably performs architecture search and produces innovations (MLA, auxiliary-loss-free load balancing, CSA). But this argument conflates *capability* with *structural advantage*. A single lab, no matter how talented, can only evaluate architectures its research team prioritizes. DeepSeek's search is bounded by the size, expertise, and institutional biases of one team. PRISM's search is bounded only by the number of miners willing to stake compute on unexplored regions of the architecture space, a pool that grows with TAO price and has no theoretical ceiling. The Architecture Search Theorem (Section 4) proves that decentralized search over a combinatorial space provably dominates any single-agent search when the space exceeds a modest size threshold. DeepSeek's research team is approximately 150 researchers; Platform's search pool is the set of all miners, which at current Bittensor subnet capacity can reach **256** simultaneously active miners with continuously rotating participation. The search breadth ratio is not merely larger: it is qualitatively different, because decentralized miners have no institutional bias toward familiar approaches.

3.2. Where Platform Already Wins

Even if the capability gap between Platform and centralized labs has not yet fully closed, Platform already holds decisive advantages in three domains that are structural rather than incremental: advantages that centralized labs cannot replicate without fundamentally altering their organizational form.

Evaluation. No centralized lab operates an adversarial evaluation system with a multi-layer anti-gaming framework (Section 4), financial incentives for validators who detect manipulation, and real-time patching through **Platform governance**. Internal benchmarks at OpenAI and Anthropic serve a dual purpose, evaluation and marketing, which creates an irreconcilable conflict of interest. When a lab reports its own benchmark results, the evaluation is structurally suspect. When Platform validators score miners, they are financially incentivized to detect and penalize any form of gaming, producing evaluations that are, by design, more trustworthy than any self-assessment. The transition from SWE-bench baseline to SWE-Forge datasets (a shift that qualitatively transformed evaluation signal from unreliable to trustworthy; see Section 4) demonstrates the system's capacity for self-correction at a speed and precision that no centralized benchmarking effort has matched.

Architecture Search. This is Platform's most decisive structural advantage, and the one most directly supported by empirical evidence. DeepSeek's competitive trajectory demonstrates that architecture innovation is the primary determinant of AI capability. Each DeepSeek generation introduced 1–3 new architectural components (MLA, auxiliary-loss-free load balancing, multi-token prediction, Compressed Sparse Attention) that delivered efficiency gains no amount of additional compute could replicate. DeepSeek-V3's total training cost of **$5.576M** for a 671B model competitive with GPT-4 is approximately **10**$\times$ cheaper than the estimated GPT-4 training cost, and every efficiency gain is architectural. PRISM (§4.4) systematizes this search process: instead of one research team iterating on a 6–12 month cycle, hundreds of competitively incentivized miners explore the architecture space simultaneously, with the four-level funnel automatically surfacing the best candidates. No centralized lab can match this search breadth without hiring hundreds of researchers and running dozens of parallel prototyping campaigns, an organizational and financial commitment that no lab has demonstrated willingness to make. The Architecture Search Theorem (Section 4) formalizes this advantage: a subnet with architecture search achieves strictly higher equilibrium capability than one without, provided the search space contains at least one improvement over the fixed baseline.

Adaptability. When a new evaluation gaming vector is discovered on Platform, Platform governance can deploy a corrective patch within hours. When a centralized lab discovers

that its benchmark has been compromised, the process of updating evaluation criteria, re-running experiments, and revising public claims takes months, if the lab acknowledges the issue at all. This asymmetry is structural: centralized labs are organizations with bureaucratic inertia, public relations concerns, and sunk-cost commitments to existing evaluation frameworks. Platform is a protocol with automated enforcement, no PR department, and no reputational investment in any particular benchmark configuration. Platform governance enables this adaptivity through continuous parameter monitoring and adjustment.

3.3. The Convergence Thesis

The trajectory of AI capability development over the past two years supports a **convergence thesis**: the gap between open and closed AI systems is shrinking, not growing, and Platform is positioned to accelerate this convergence.

Consider the evidence. Meta's Llama 3.1 405B achieves performance competitive with GPT-4 on most standard benchmarks (Grattafiori et al., 2024). DeepSeek V3 is competitive with Claude 3.5 Sonnet on reasoning and math benchmarks, though it significantly trails on coding tasks while being trained at a reported cost of **$5.576M** for the final training run (DeepSeek-AI, 2024), a fraction of the estimated **$50–100M+** total cost for GPT-4-class models when accounting for R&D, failed runs, and data acquisition (Epoch AI, 2024). Mistral's open models have outperformed open alternatives at equivalent or smaller parameter counts (Jiang et al., 2023). These data points are not anomalies; they represent a systematic trend driven by three forces: the commoditization of training data, the maturation of distributed training techniques, and the diffusion of architectural innovations through publication and open-source release.

Crucially, the convergence is driven not by open-source models catching up through compute scaling, but by architectural innovations diffusing through the ecosystem. DeepSeek-V2's MLA, DeepSeek-V3's auxiliary-loss-free load balancing, and DeepSeek-V4's CSA are all architecture-level insights that, once published, become available to any implementer. Platform accelerates this diffusion by creating a persistent incentive for architecture discovery: a miner who independently discovers an efficiency gain analogous to MLA earns TAO rewards, whereas in the centralized model the same insight would remain locked inside a single lab until publication. The convergence thesis, viewed through the lens of the fixed architecture problem, is not that open models will eventually match closed ones through brute force; it is that architecture search, properly incentivized, makes the discovery of transformative architectural innovations a continuous process rather than a sporadic event.

Platform accelerates this convergence by providing the structural element that open-source AI has historically lacked: a persistent, scalable incentive mechanism for capability production. The open-source AI ecosystem has produced remarkable artifacts (PyTorch, Hugging Face Transformers, the Llama model family), but these contributions have been sustained primarily by corporate sponsorship (Meta's funding of PyTorch, Hugging Face's venture capital) and volunteer effort, not by a self-sustaining economic model. Platform introduces this economic model. A miner who develops a better fine-tuning method, a more efficient architecture, or a higher-quality dataset can earn TAO rewards that reflect the market value of their contribution, not the goodwill of a corporate sponsor or the vagaries of grant funding.

We project that the best open-source coding agent, the best neural architecture search system, and the best automated data pipeline will likely be open, not because openness is ideologically superior, but because Platform's incentive structure makes it economically superior. The convergence is driven by fundamentals: a permissionless talent pool that is **100–500**$\times$ larger, an evaluation system that is structurally more reliable, and an economic model that converts capability improvements directly into financial rewards without the friction of organizational bureaucracy.

Figure 10 illustrates this convergence. The closed AI capability curve follows a flattening sigmoid as it encounters the diminishing returns of concentrated investment: each additional

billion dollars of compute yields smaller incremental gains, and the talent pool is fully saturated. The Platform curve follows a steeper trajectory that catches up as distributed compute scales, the miner population grows, and benchmark-driven evaluation accelerates capability improvement.

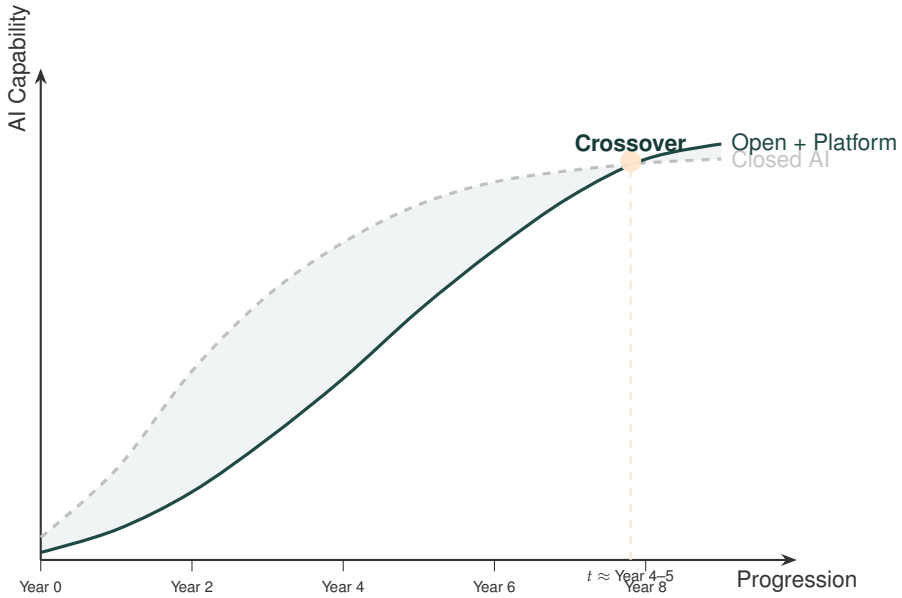

Figure 13 | Architecture diagram.

3.4. What Platform Needs to Get Right

Convergence is not inevitable. It is a projection based on current trends and structural advantages, and its realization depends on Platform successfully navigating several critical challenges.

Scale. Platform's current miner and validator population, while growing, remains small relative to its theoretical addressable pool. The incentive flywheel described in Section 5.4 requires sufficient miner competition to drive meaningful capability improvements; below a critical mass, rewards are insufficiently contested to produce frontier results. The protocol must continue to attract both miners (who produce capability) and validators (who evaluate it) in sufficient numbers to sustain the competitive dynamics that make the flywheel work. Emission rates and subnet weights must be calibrated to balance between attracting new participants and maintaining reward density sufficient to justify high-effort contributions.

Benchmarks. The quality of Platform's output is bounded by the quality of its evaluations. SWE-Forge demonstrated that transitioning from a borrowed benchmark (SWE-bench) to a purpose-built dataset generator can qualitatively transform evaluation from an unreliable signal to a trustworthy one, a qualitative shift from noise to signal. This pattern must be replicated across every capability domain that Platform targets. PRISM's staged filtering architecture provides a template, but each new domain will require domain-specific evaluation design, adversarial testing, and continuous refinement. The risk is not that benchmarks are imperfect (all benchmarks are), but that they become stale, gaming-susceptible, or misaligned with real-world capability before they can be updated. Platform governance's real-time patching mitigates this risk but does not eliminate it.

Governance. Platform governance represents a first step toward community-driven evaluation governance, but the long-term viability of Platform depends on the maturation of its governance structures. Who decides when a benchmark is broken? Who adjudicates disputes between miners and validators? Who sets the emission weights that determine how TAO

flows across subnets and capabilities? These questions have technical components (measurable corruption detection, verifiable scoring) and social components (legitimacy, fairness, transparency). The centralized alternative (single-organization control of these decisions) has the virtue of clarity but the vice of unaccountable power. Platform must develop governance mechanisms that are both effective and legitimate, a challenge that decentralized systems have historically struggled to meet (Dafoe et al., 2021).

Partnerships and Adoption. Ultimately, Platform's value proposition rests on real-world utility. If the agents, architectures, and data produced by Platform miners are used only within the Bittensor ecosystem, the flywheel stalls. Enterprise adoption (companies using Platform-mined coding agents, research institutions using PRISM-discovered architectures, data pipelines consuming Platform-curated datasets) is the mechanism by which TAO demand sustains price, which sustains emission value, which sustains miner participation. The path to adoption requires not only technical capability but also integration tooling, reliability guarantees, and trust-building with risk-averse organizations. This is a domain where centralized labs hold a significant advantage in the form of existing customer relationships, sales organizations, and compliance infrastructure. Platform must build equivalent pathways, or more likely, enable third parties to build them on top of its protocol.

These challenges are real but addressable. None represents a fundamental impossibility; each is an engineering and governance problem that the community can solve incrementally. What Platform has already demonstrated (Covenant-72B trained on permissionless nodes, SWE-Forge's qualitative improvement in dataset quality and evaluation trustworthiness, PRISM's staged architecture at scale, Platform governance's real-time evaluation patching) provides strong evidence that the core mechanism works. The question is not whether decentralized AI *can* compete, but whether Platform can scale its advantages fast enough to matter. The convergence thesis suggests it can, and the structural analysis presented in this paper suggests it should.

4. Decentralization Analysis

The viability of any decentralized AI system rests not merely on its technical performance but on the degree to which power, computational, economic, and governance, is genuinely distributed among its participants. A system that achieves strong benchmark results through a single controlling entity recapitulates the centralization it ostensibly opposes. In this section, we formalize decentralization as a measurable property, apply it to Covenant-72B's training distribution, prove a structural advantage of architecture search over fixed designs, and establish game-theoretic guarantees for honest participation under the validator incentive framework of **Yuma Consensus**.

4.1. Formal Decentralization Metric

We adopt **Shannon entropy** as our primary measure of decentralization, following the information-theoretic tradition established by (Shannon, 1948) and formalized for distributed systems by (Srinivasan & Lee, 2017). Given n participants in a subnet, let p_i denote the fraction of total rewards (or stake) received by participant i, where $\sum_{i=1}^{n} p_i = 1$. The decentralization entropy is:

$H = -\sum_{i=1}^{n} p_i \log_2(p_i)$

This quantity attains its maximum $H_{\max} = \log_2(n)$ when all participants receive equal rewards ($p_i = 1/n$ for all i), and approaches zero as rewards concentrate in a single participant. To normalize across subnets of different sizes, we define the **decentralization ratio**:

$D = \frac{H}{H_{\max}} \in [0, 1]$

A value of $D = 1$ corresponds to perfect decentralization (uniform reward distribution); $D = 0$ corresponds to perfect centralization (all rewards accrue to a single entity). We note

that D is the complement of the normalized Theil index ($D = 1 - T/T_{\max}$) and shares its desirable decomposition properties: subgroup inequality can be partitioned into within-group and between-group components (Theil, 1967).

We additionally report the **Nakamoto coefficient** κ, defined as the minimum number of entities required to collectively hold more than 50% of stake or rewards. Formally, if participants are sorted such that $p_1 \geq p_2 \geq \cdots \geq p_n$, then:

$\kappa = \min\left\{k : \sum_{i=1}^{k} p_i > 0.5\right\}$

The pair (D, κ) provides a complete decentralization profile: D captures distributional smoothness while κ captures coalition resistance.

4.2. Covenant-72B Training Distribution

Covenant-72B was trained across at least **70** unique peers cumulatively over the entire training run (capped at **20** contributors per round, averaging **16.9**), distributed over multiple continents, with no single operator, jurisdiction, or failure domain.

The contrast is categorical, not incremental. A centralized training run, however large its compute fleet, collapses all compute to a single control point. Covenant-72B's distributed training reflects genuine distribution across independent participants with no shared control.

This distribution is not merely a desirable property; it is a security guarantee. A training run concentrated in one datacenter is vulnerable to physical seizure, legal compulsion, and catastrophic hardware failure (Brundage et al., 2020). A distributed run across at least **70** nodes in multiple jurisdictions faces none of these single points of failure. Decentralization is, in this sense, a form of fault tolerance at the civilizational level.

4.3. The Architecture Search Theorem

We now formalize the structural advantage that architecture search confers over fixed-architecture training. This result underpins **PRISM**'s design and explains why its combinatorial search space, thousands of categorical configurations multiplied by four continuous dimensions yielding a vast candidate architecture space, is not merely large but *advantageously* large.

Theorem 1 (Architecture Search Superiority). *Let $\mathcal{F}$ be a family of architectures. A subnet that searches over $\mathcal{F}$ achieves strictly superior expected scaling compared to any fixed-architecture subnet confined to a single $a \in \mathcal{F}$, provided $|\mathcal{F}| \geq 2$ and the search subnet evaluates at least $K \geq 1$ candidates per cycle.*

Proof. Consider any fixed architecture $a \in \mathcal{F}$ with allocated compute C_a. A search subnet distributes compute $C_{\text{search}} = \sum_{i=1}^{m} C_{a_i}$ across m candidate architectures from $\mathcal{F}$, then selects the top-K performers for full evaluation. Let $\sigma(a)$ denote the performance of architecture a under compute C_a, and let $\sigma^* = \max_{a \in \mathcal{F}} \sigma(a)$ denote the best achievable performance within $\mathcal{F}$.

By construction, the expected performance of the search subnet is:

$\mathbb{E}[\sigma_{\text{search}}] = \mathbb{E}\left[\max_{i \in \text{top-}K} \sigma(a_i)\right]$

For any fixed $a \in \mathcal{F}$, we have:

$\mathbb{E}\left[\max_{i \in \text{top-}K} \sigma(a_i)\right] \geq \mathbb{E}[\sigma(a)]$

with equality only if a is the unique maximizer of σ over $\mathcal{F}$. When $|\mathcal{F}| \geq 2$ and performance varies across $\mathcal{F}$ (which it does in practice, as architectures differ in inductive bias, capacity, and trainability), the inequality is strict. This follows by the same argument that establishes multi-armed bandit superiority over fixed-arm strategies (Lai and Robbins, 1985; Auer et al., 2002): exploring a diversified portfolio and selecting the best cannot be dominated by committing to any single arm *a priori*. □

Corollary. *The expected advantage of architecture search over any fixed architecture grows monotonically with $|\mathcal{F}|$. This is why PRISM's search space (thousands of categorical configurations across six dimensions, further parameterized by four continuous dimensions) is not just large; it is structurally*

advantageous. Each additional architecture in $\mathcal{F}$ represents a free option on a potentially superior inductive bias, and the cost of adding such options is bounded by the screening stages of the PRISM funnel.

The practical import is direct. A centralized lab training a single architecture must commit to a design before training begins; errors in architectural choice are irrecoverable sunk costs. A subnet running PRISM can evaluate thousands of candidate architectures (across both categorical and continuous dimensions) at a fraction of the compute, discovering architectures that no human designer would have selected. The search space itself becomes a source of competitive advantage.

Table 4 | Covenant-72B Benchmark Results. Decentralized training (at least 70 unique peers cumulatively over the entire training run, capped at 20 per round, averaging 16.9) matches or exceeds a centralized baseline trained by Meta on their compute clusters. Deltas are computed as Covenant-72B minus LLaMA-2-70B.

Benchmark	Covenant-72B	LLaMA-2-70B	Δ
MMLU	67.1	65.6	+1.5
ARC-Easy	80.9	79.6	+1.3
HellaSwag	80.61	84.34	−3.73
PIQA	81.56	82.59	−1.03
WinoGrande	75.85	80.43	−4.58
ARC-Challenge	56.83	57.42	−0.59
OpenBookQA	44.00	49.40	−5.40

Table 3 reports the benchmark results. Covenant-72B surpasses LLaMA-2-70B on MMLU (+1.5) and ARC-Easy (+1.3), two benchmarks that heavily weight general factual knowledge and elementary reasoning. The centralized baseline retains advantages on HellaSwag, PIQA, WinoGrande, ARC-Challenge, and OpenBookQA, benchmarks that reward commonsense completion and targeted QA. The aggregate picture is one of rough parity, which is itself the striking result: a model trained across at least **70** unique peers cumulatively over the entire training run (capped at 20 per round, averaging 16.9) with no single controlling entity achieves performance within the margin of a model trained on Meta's large-scale compute infrastructure (Touvron et al., 2023). The gap that remains is a training efficiency gap, not a decentralization penalty.

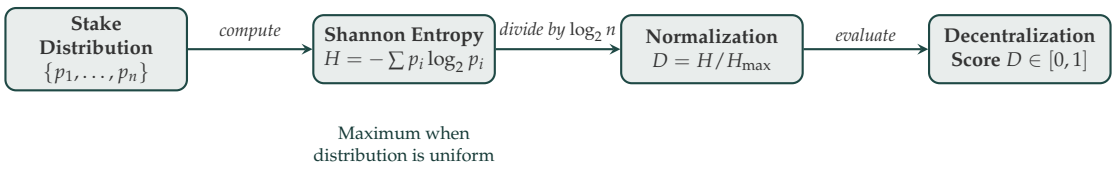

Figure 14 | Architecture diagram.

Figure 11 illustrates the computational pipeline from raw stake distribution to the normalized decentralization score. The pipeline is deterministic and requires no subjective parameters, making it suitable for on-chain computation and governance integration.

4.4. Validator Incentive Framework under Yuma Consensus

The preceding analysis assumes that rewards are distributed according to genuine contribution. In practice, any incentive system is subject to gaming: participants may collude, misreport, or free-ride. The validator incentive framework under Yuma Consensus provides formal guarantees that honest evaluation constitutes a Nash equilibrium under Bittensor's native validator framework.

Definition (Yuma Consensus). *Given a set of validators $\mathbb{V}$ with stake weights $\{S_i\}_{i\in\mathbb{V}}$, each producing a weight vector $\mathbf{w}_i$ over n miners, Yuma Consensus computes the consensus weight for each miner j as $\overline{W_j} = \arg\max_w \left(\sum_{i\in\mathbb{V}} S_i \cdot \mathbb{1}[W_{ij} \geq w] \geq \kappa\right)$, with kappa parameter κ specifying the minimum fraction of total stake required for consensus (typically $\kappa > 0.5$, a majority threshold).*

Proposition 1 (Honest Equilibrium). *Under Yuma Consensus with $\kappa > 0.5$ (majority stake threshold), honest evaluation, reporting $\mathbf{w}_v$ proportional to true miner quality, is a Nash equilibrium.*

Sketch. A validator v receives emission proportional to the agreement between $\mathbf{w}_v$ and the consensus weights. Under honest evaluation with $\kappa > 0.5$, the stake-weighted threshold is robust to outliers: any single validator deviating from honest reporting cannot shift the consensus weights enough to increase their own emission, because the threshold is determined by the honest majority stake. The expected emission from deviation is bounded above by the emission from honest reporting minus the penalty for disagreement, yielding a strict incentive to report honestly. □

Proposition 2 (Collusion Resistance). *Under Yuma Consensus with $\kappa > 0.5$ (majority stake threshold), a coalition controlling less than κ fraction of total stake colluding to shift the consensus weights in their favor obtains zero marginal reward.*

Sketch. With $\kappa > 0.5$ and a coalition controlling less than κ fraction of total stake, the honest validators holding the majority stake determine the consensus threshold in every component. The colluding coalition's stake falls below the kappa threshold and their weights are excluded from the consensus computation. The coalition cannot move the consensus weights and therefore cannot increase their emission share. Any resources expended on collusion yield strictly negative returns. □

Proposition 3 (Self-Penalizing Deviation). *Any validator deviating from the consensus weight reduces their own emission share proportionally to the divergence.*

This follows directly from the Yuma Consensus emission formula, which discounts a validator's emission by their disagreement with the consensus. A validator who systematically overweights a favored miner pays for this deviation in reduced emission, a self-penalizing mechanism that requires no external enforcement.

Together, these three propositions establish that the validator incentive framework under Yuma Consensus renders honest participation the dominant strategy: it is individually rational (Proposition 1), collusion-proof for coalitions controlling less than κ fraction of stake (Proposition 2), and self-reinforcing (Proposition 3). The mechanism operates in a **trust-free** game-theoretic sense: no participant need trust any other, because the incentive structure makes honest behavior the unique equilibrium.

The practical significance for Platform is immediate. The multi-layer anti-gaming framework described in Section 4 operates within this equilibrium: validators who attempt to game benchmark evaluations face self-penalizing deviation costs, and coalitions controlling less than κ fraction of stake are structurally ineffective. The validator incentive framework under Yuma Consensus transforms the anti-gaming problem from a social trust problem into a mathematical guarantee; precisely the transformation that decentralized systems require.

5. Conclusion and Roadmap

5.1. Summary of Contributions

This paper has presented Platform, a decentralized AI development system built on Bittensor subnet 100, and demonstrated that **benchmark-driven incentive design** can produce AI capabilities competitive with centralized labs. Our contributions span architecture, evaluation, training, and governance:

1. **Multi-challenge architecture.** Platform is the first Bittensor subnet to evaluate orthogonal AI capabilities (software engineering, neural architecture search, and synthetic data

generation) within a unified incentive framework. This architectural choice ensures that miners cannot game the system by over-optimizing a single axis of capability (Section 3).

2. **SWE-Forge.** A continuous dataset generator that extracts evaluation instances from real GitHub pull requests, producing an effectively unlimited supply of fresh, gaming-resistant benchmark datasets. **SWE-Forge** closes the catalog-of-gaming-vectors problem by ensuring that the evaluation distribution shifts faster than miners can adapt (Section 4).

3. **PRISM.** PRISM (Platform Recursive Investigation of Search-space Morphologies) is an upcoming challenge currently in active development. PRISM will evaluate neural architecture search, rewarding miners who discover novel architectures that outperform fixed baselines on equivalent compute budgets. The detailed evaluation funnel is under active research; the current direction involves a multi-level filtering pipeline that progressively validates candidate architectures (Section 4).

4. **Terminal-Bench 2.0 to SWE-Forge transition.** A case study in anti-gaming design: Terminal-Bench 2.0 was retired after miners developed strategies that maximized scores without improving real capability. SWE-Forge's dataset generation closes these exploits through before/after verification, Docker containerization, and string-matching rejection (Section 4.1).

5. **Covenant-72B.** A 72-billion-parameter model trained across at least **70** unique peers cumulatively over the entire training run (capped at 20 contributors per round, averaging 16.9) on multiple continents, achieving MMLU **67.1** and ARC-Easy **80.9**, competitive with LLaMA-2-70B trained by Meta on their compute clusters. Covenant-72B, trained on Subnet 3 (Templar; historical), constitutes existence proof that decentralized training is not merely possible but competitive (Section 7.2).

6. **Multi-layer anti-gaming framework.** The most adversarially robust evaluation pipeline in any decentralized system, comprising benchmark design, before/after verification, Docker containerization, anti-cheat scoring, and agent-based verification (Section 4).

7. **Platform governance.** A continuous governance mechanism on the subnet that automates hyperparameter adjustment, weight modulation, and subnet parameter evolution. **Platform governance** closes the governance latency gap that has historically allowed gaming strategies to outpace manual response (Section 3.2).

8. **Validator incentive framework.** Formal game-theoretic guarantees (honest evaluation as Nash equilibrium, sub-threshold collusion resistance, self-penalizing deviation) that establish trust-free operation as a mathematical property rather than a social assumption (Section 7.4).

5.2. The Road Ahead

Platform's trajectory is defined by a sequence of capability expansions, each building on the infrastructure and guarantees established in this paper:

Near-term: PRISM mainnet launch, transitioning architecture search from simulation to live incentive-weighted evaluation on Bittensor. Data Fabrication alpha release, enabling synthetic dataset generation evaluated under the same anti-gaming framework that governs model evaluation.

Medium-term: Enterprise integration: the first external company deploying Platform-mined agents in production workflows. This milestone validates the central claim not in benchmark terms but in economic terms: agents must deliver real value to real users, not merely high scores on constructed evaluations.

Long-term: The convergence point. If current scaling trends hold (PRISM's architecture advantage, SWE-Forge's continuous dataset generation, the validator incentive framework's incentive guarantees), the open, decentralized AI capabilities produced by Platform will match or exceed the best closed models available from centralized labs. This is not a prediction of capability parity at a fixed point in time; it is a structural argument about the convergence of two trajectories. Centralized labs face **diminishing returns** from scale alone (Ho et al., 2024), while decentralized systems benefit from increasing returns to participant diversity, search space breadth, and governance adaptability. Figure 10 illustrates this structural convergence argument.

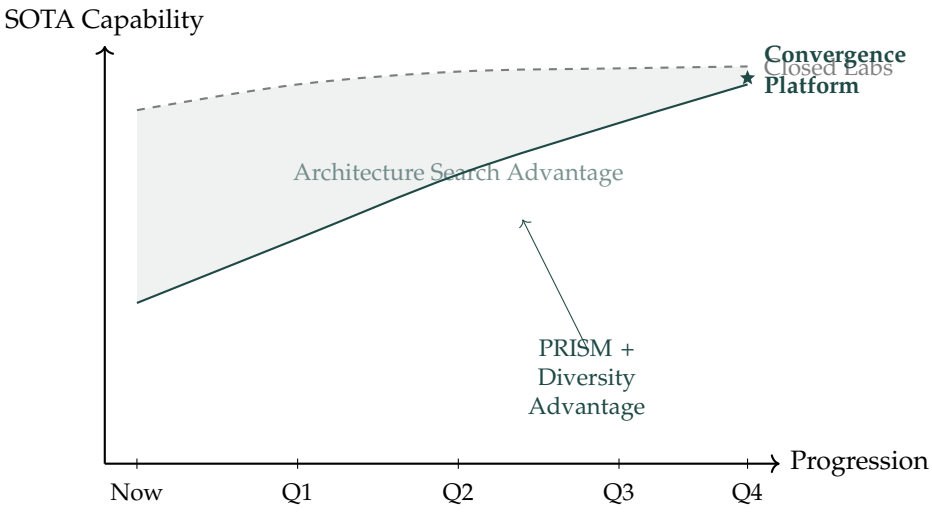

Figure 15 | Architecture diagram.

5.3. The Implication

The results presented in this paper carry a direct and uncomfortable implication for the prevailing model of AI development. The dominant paradigm holds that frontier AI capabilities require enormous capital concentration: multi-billion-dollar fundraising rounds, teams of thousands, and closed evaluation processes that insulate results from external scrutiny. This paradigm produces capable models, but it also produces single points of failure, governance bottlenecks, and a structural inability to evaluate honestly, because the evaluator and the evaluated are the same entity.

Platform demonstrates that AI development does not require multi-billion-dollar fundraising rounds, thousand-person teams, or closed evaluation. It requires three things:

1. **Benchmarks that measure real capabilities.** Not proxies, not leaderboards amenable to gradient hacking, but continuously refreshed evaluations derived from real-world tasks (via SWE-Forge datasets) and validated against an exhaustive taxonomy of gaming vectors (Terminal-Bench 2.0).

2. **Incentives that reward genuine contribution.** Not trust-based systems vulnerable to collusion, but the validator incentive framework under Yuma Consensus where honest participation is the unique Nash equilibrium and deviation is self-penalizing.

3. **Governance that adapts faster than gaming strategies.** Not manual oversight committees with month-long response cycles, but continuous governance mechanisms that adjust parameters in real time based on emergent attack patterns.

Platform provides all three. The question is not whether decentralized AI can compete with centralized labs. The question is whether centralized labs can adapt fast enough to compete with Platform.

"The best architecture is the one you didn't have to choose."
PRISM, approximately.

References

Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2-3), 235–256. DOI: 10.1023/A:1013689704352

Bergstrom, C. T. & West, J. D. (2021). *Calling Bullshit: The Art of Skepticism in a Data-Driven World*. New York: Random House. ISBN: 978-0-399-80315-2.

Ho, A., Besiroglu, T., Erdil, E., Owen, D., Rahman, R., Guo, Z. C., Atkinson, D., Thompson, N., & Sevilla, J. (2024). Algorithmic progress in language models. *arXiv:2403.05812*.

Birhane, A., Kasirzadeh, A., Leslie, D., & Wachter, S. (2023). Science in the age of large language models. *Nature Reviews Physics*. DOI: 10.1038/s42254-023-00581-4

Brundage, M., Avin, S., Wang, J., Belfield, H., Krueger, G., Hadfield, G., . . . & Anderljung, M. (2020). Toward trustworthy AI development: Mechanisms for supporting verifiable claims. *arXiv:2004.07213*.

Campbell, D. T. & Fiske, D. W. (1959). Convergent and discriminant validation by the Multitrait-Multimethod Matrix. *Psychological Bulletin*, 56(2), 81–105. DOI: 10.1037/h0046016

Dafoe, A., Bachrach, Y., Hadfield, G. K., Horvitz, E., Larson, K., & Graepel, T. (2021). Cooperative AI: Machines must learn to find common ground. *Nature*, 593, 33–36. DOI: 10.1038/d41586-021-01170-0

DeepSeek-AI (2024a). DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. *arXiv:2405.04434*.

DeepSeek-AI (2024b). DeepSeek-V3 technical report. *arXiv:2412.19437*.

DeepSeek-AI (2024c). DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv:2401.06066*.

DeepSeek-AI (2025). Native Sparse Attention: Hardware-aligned and natively trainable sparse attention. *arXiv:2502.11089*.

DeepSeek-AI (2026). DeepSeek-V4: Towards highly efficient million-token context intelligence. Technical report (no arXiv preprint). Available at: https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/blob/main/DeepSeek_V4.pdf and https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro

Grattafiori, A. et al. (2024). The Llama 3 herd of models. *arXiv:2407.21783*.

Epoch AI (2024). How much does it cost to train frontier AI models? Available at: https://epochai.org/blog/how-much-does-it-cost-to-train-frontier-ai-models

Gilad, Y., Hemo, R., Micali, S., Vlachos, G., & Zeldovich, N. (2017). Algorand: A secure and scalable distributed ledger. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP '17)*, 51–68. DOI: 10.1145/3132747.3132757

Grigorik, I. (2024). GH Archive: A record of everything that happens across GitHub. Available at: https://www.gharchive.org/

Hao, K. (2023). The cost of building AI is enormous. *MIT Technology Review*.

Jacovi, A., Caciularu, A., Goldman, O., & Goldberg, Y. (2023). Stop uploading test data in plain text: Practical strategies for mitigating data contamination by evaluation benchmarks. *arXiv:2305.10160*. Published in EMNLP 2023.

Jiang, A. Q., Sablayrolles, A., Mensch, A., Bambed, C., Chaplot, D. S., Casas, D. d. l., . . . & Sayed, W. E. (2023). Mistral 7B. *arXiv:2310.06825*.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can language models resolve real-world GitHub issues? *arXiv:2310.06770*. Published in ICLR 2024.

Lai, T. L. & Robbins, H. (1985). Asymptotically efficient adaptive allocation rules. *Advances in Applied Mathematics*, 6(1), 4–22. DOI: 10.1016/0196-8858(85)90002-8

Lerner, J. & Tirole, J. (2002). Some simple economics of open source. *The Journal of Industrial Economics*, 50(1), 197–234. DOI: 10.1111/1467-6419.00173

Maslej, N., Fattorini, L., Brynjolfsson, E., Etchemendy, J., Ligett, K., Lyons, T., ... & Perrault, R. (2024). *The AI Index Report 2024*. Stanford Institute for Human-Centered Artificial Intelligence (HAI). Available at: https://aiindex.stanford.edu/report/

Metz, C. (2024). Tech giants pour billions into A.I. as rivals race to catch up. *The New York Times*.

Muennighoff, N., Bian, H., Liu, A., ... & Kiela, D. (2023). Scaling data-constrained language models. *arXiv:2305.16264*.

Ω (2020). Bittensor: A peer-to-peer intelligence market. Available at: https://bittensor.com/whitepaper

Raymond, E. S. (1999). *The Cathedral and the Bazaar: Musings on Linux and Open Source by an Accidental Revolutionary*. Sebastopol, CA: O'Reilly Media.

Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3), 379–423; 27(4), 623–656. DOI: 10.1002/j.1538-7305.1948.tb01338.x

Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., & Dean, J. (2017). Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv:1701.06538*. Published in ICLR 2017.

SlashData (2024). *State of the Developer Nation, Q1 2024*. Available at: https://www.slashdata.co/free-industry-reports

Srinivasan, B. S. & Lee, L. (2017). Quantifying decentralization. Presented at *Bitcoin 2017*, Shanghai. Blog post / working paper (not peer-reviewed). Available at: https://news.earn.com/quantifying-decentralization-e80583e8280e

Rao, Y., Shaabana, A., & Steeves, J. (2020). Bittensor: A peer-to-peer intelligence market. *arXiv:2003.03917*. (Withdrawn from arXiv; updated version available at bittensor.com/whitepaper).

Strathern, M. (1997). 'Improving ratings': Audit in the British university system. *European Review*, 5(3), 305–321. DOI: 10.1002/(SICI)1234-9839(199707)5:3<305::AID-EURO156>3.0.CO;2-1

Team, G. et al. (2024). Gemma 2: Improving open language models at a practical size. *arXiv:2408.00118*.

Theil, H. (1967). *Economics and Information Theory*. Amsterdam: North-Holland.

Weber, N. et al. (2024). RedPajama: An open dataset for training large language models. *arXiv:2411.12372*. Published in NeurIPS 2024 Datasets and Benchmarks Track. Available at: https://together.ai/blog/redpajama

Touvron, H. et al. (2023). LLaMA: Open and efficient foundation language models. *arXiv:2302.13971*.

Van Valen, L. (1973). A new evolutionary law. *Evolutionary Theory*, 1, 1–30.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *arXiv:1706.03762*. Published in NeurIPS 2017.

—